

Le Curve di Bezier

Computer Graphics 2025/2026

Problema :

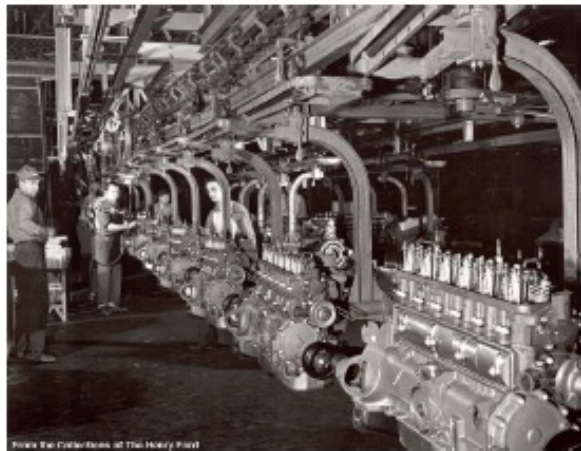
(Ford 1947, Fiat 1955, Citroen 1959, Renault 1960):
automazione del processo industriale nella
progettazione/produzione delle automobili



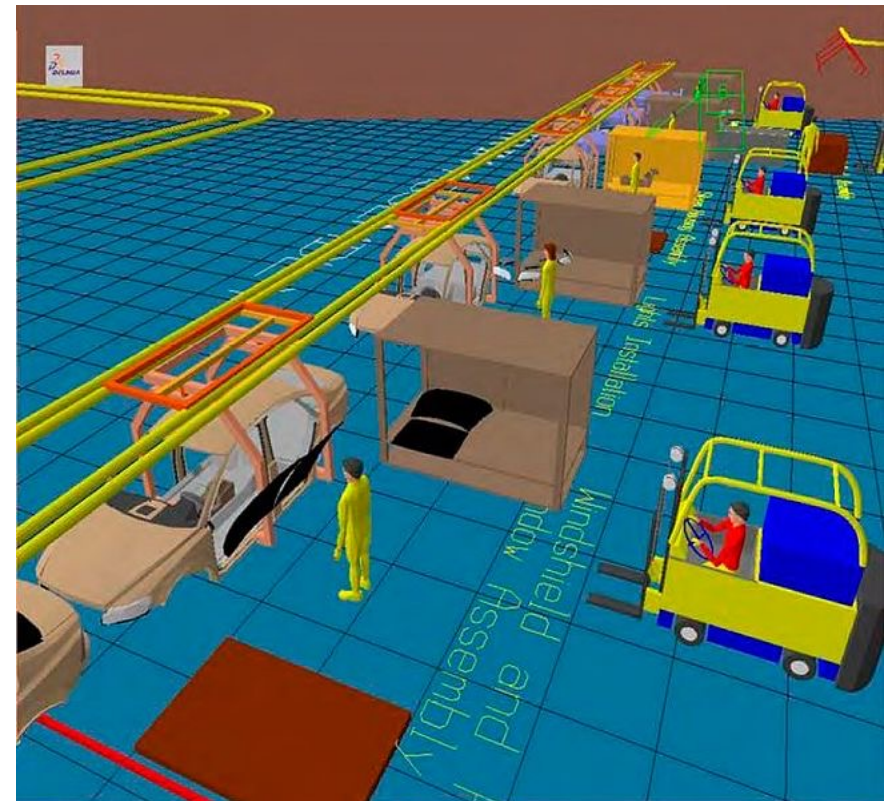
FIAT 600

Siamo nel periodo della storia compreso tra gli anni cinquanta e settanta
del XX secolo di forte crescita industriale

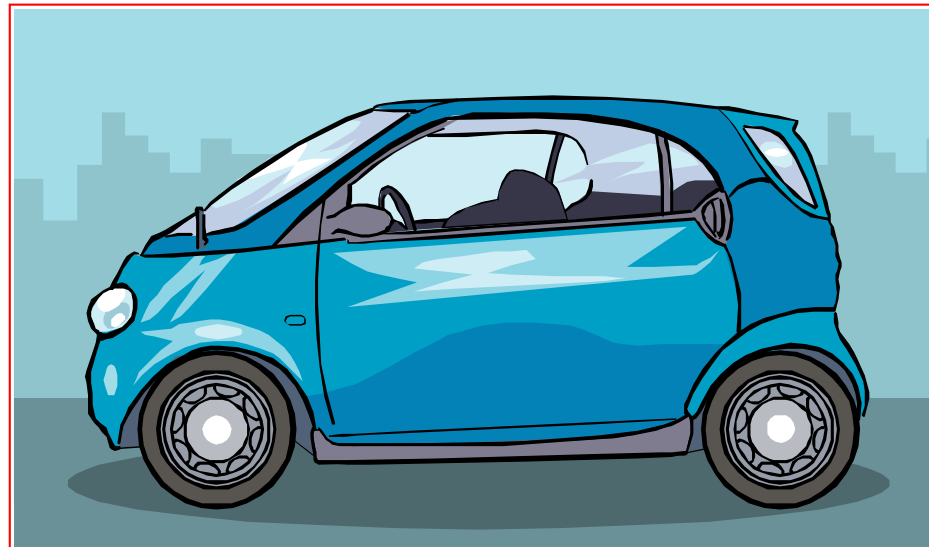
Il termine **automation** fu introdotto nel 1947 presso la Ford Motor Company come contrazione dell'espressione "automatic production" per indicare l'insieme di apparati di movimentazione automatica che erano stati installati nelle loro linee di produzione.



Il problema delle case automobilistiche
in quegli anni riguardava la
gestione automatica delle macchine
nelle catene di montaggio
per tranciare i pezzi di lamiera per
carrozzeria
seguendo un traiettoria predefinita
(Automazione del Processo di
Produzione Industriale)



PROBLEMA MATEMATICO:
disegno della curva 2D/3D che rappresenta il profilo dell'
automobile



Come tracciare la traiettoria ?

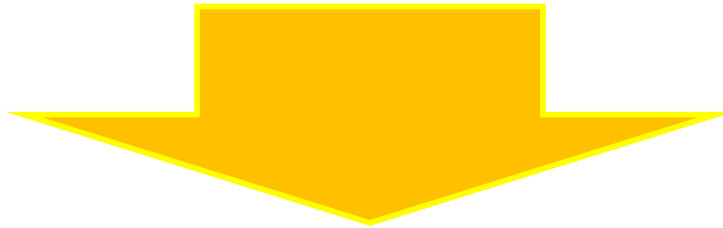


- **1950** : dai disegni costruiti a mano (plotter) a procedure automatiche
 - *de Boor* usò le **B-spline** (introdotte da **Shoenberg** nel **1946**) alla **General Motors**.
- **1959** : *de Casteljau* (matematico) fu assunto alla **Citroen (Francia)** per la costruzione di modelli a partire dai disegni. Egli usò **polinomi di Bernstein** + punti di controllo (di fatto le curve di Bezier). Il suo lavoro fu tenuto segreto fino agli anni '80.
- **1960**: *Bezier* (ingegnere) fu assunto alla **Renault (Francia)** per lo sviluppo di procedure automatiche per la costruzione di modelli. Introdusse le *curve di Bezier*.
- **1980**: alla **Boeing** (U.S.A.) si passò dalle curve di Bezier alle NURBS

Come tracciare la curva /
traiettoria ?

PROBLEMA:

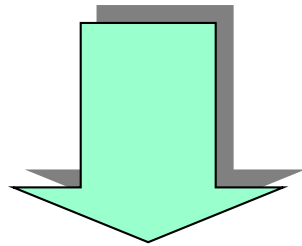
costruire **un modello matematico** che
descriva il profilo dell'auto in modo
attendibile



Tale problema rientra tra quelli della
RAPPRESENTAZIONE DI DATI

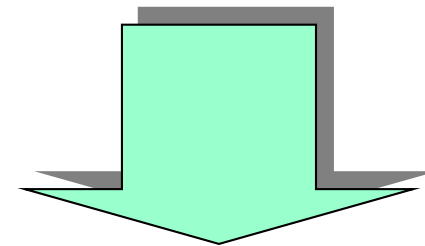
RAPPRESENTAZIONE DATI

costruire un modello che descriva dei
dati in modo attendibile



si costruisce
una curva che passa
per i punti assegnati

Modello
INTERPOLANTE

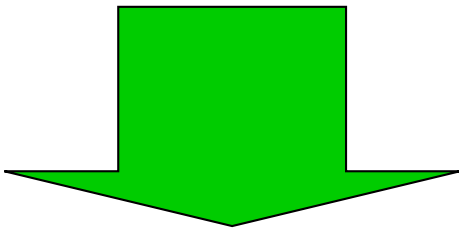


si costruisce
una curva che si avvicina
ai punti assegnati

Modello
APPROSSIMANTE 9

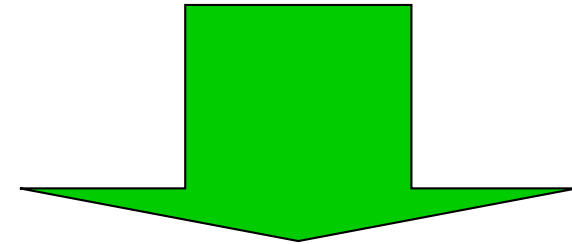
IN GENERALE

Interpolazione



Modello che assume
trascurabile l'errore nei dati

Approssimazione



Modello che assume **non**
trascurabile l'errore nei dati

Problema generale di INTERPOLAZIONE

Dati n punti distinti $(x_i, y_i)_{i=1, \dots, n}$
si vuole costruire una funzione $f(x)$
tale che nei nodi $(x_i)_{i=1, \dots, n}$
soddisfi certe condizioni, dette

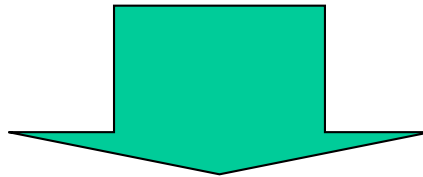
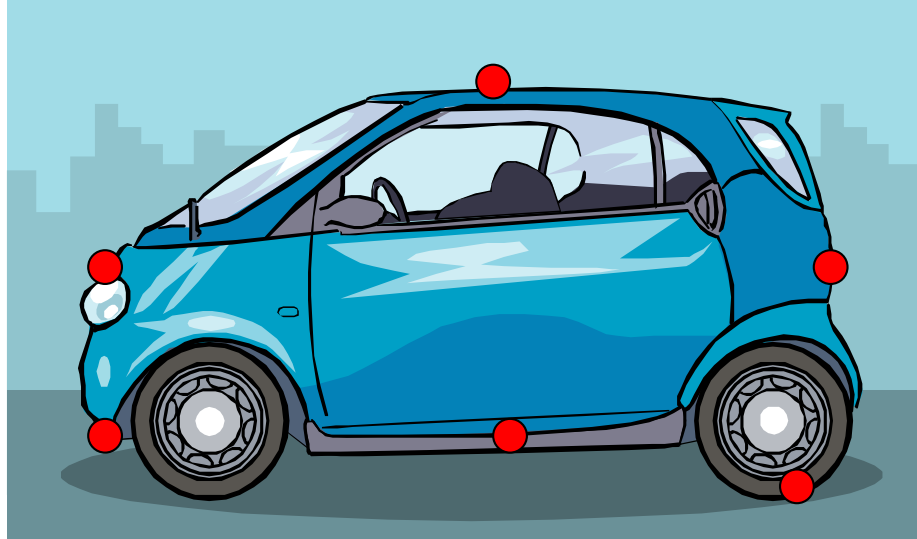
Condizioni di interpolazione

Le condizioni di interpolazione sono, in generale, vincoli
che la funzione interpolante $f(x)$ (e/o le sue derivate)
deve soddisfare nei nodi $(x_i)_{i=1, \dots, n}$

Problema generale di APPROSSIMAZIONE

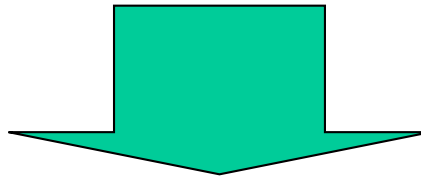
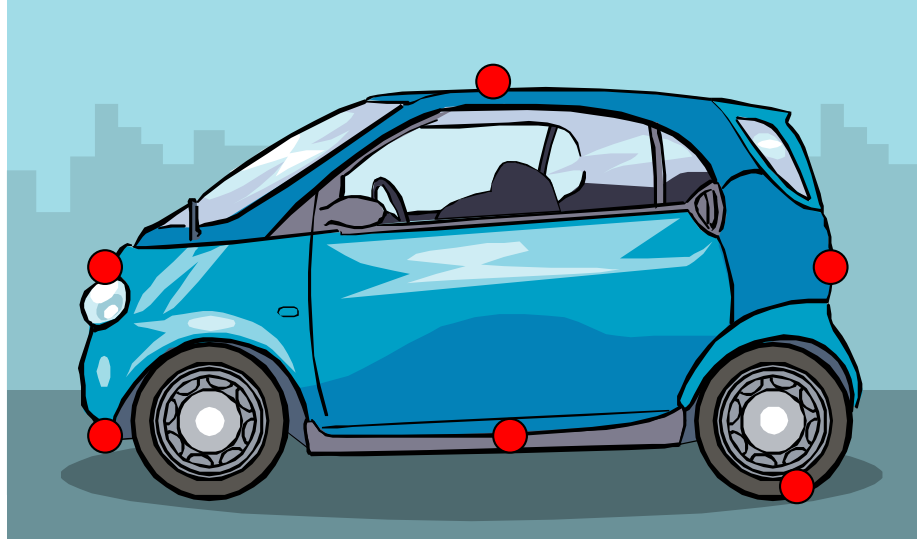
Dati n punti distinti $(x_i, y_i)_{i=1, \dots, n}$ si vuole
determinare una funzione $f(x)$
tale che nei nodi $(x_i)_{i=1, \dots, n}$
assuma i valori $(y_i)_{i=1, \dots, n}$
oppure si scosti poco da essi

Esempio: disegno del profilo di una automobile



fissiamo i punti per i quali deve
passare la curva

Esempio: disegno del profilo di una automobile



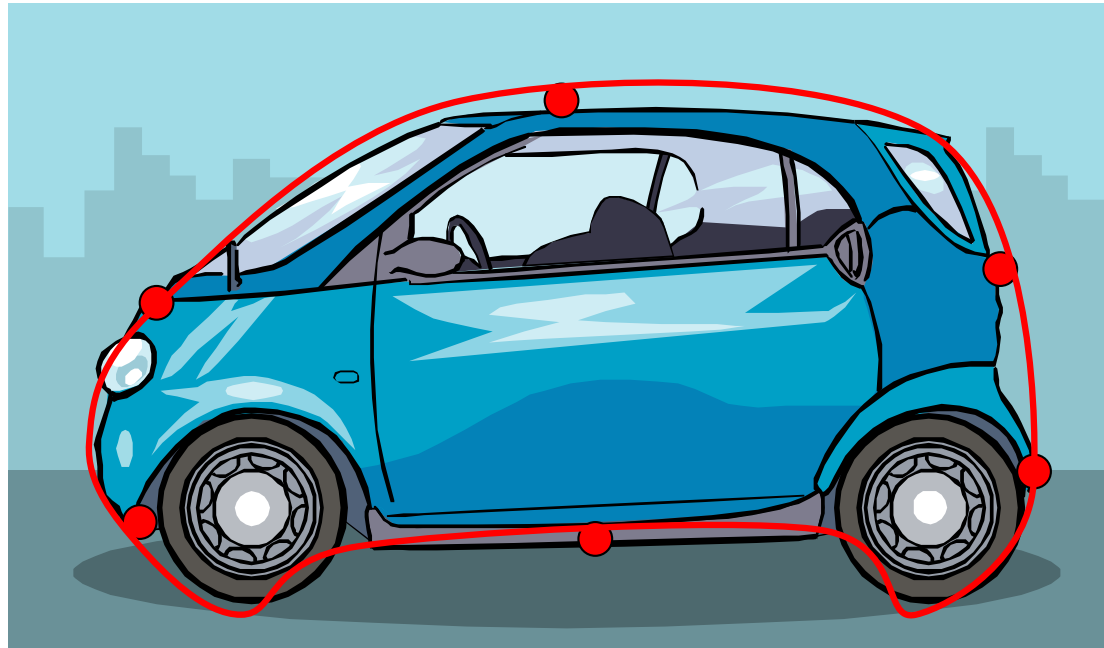
PROBLEMA DI INTERPOLAZIONE:
costruire una curva interpolante

Problema generale di INTERPOLAZIONE

Dati n punti distinti $(x_i, y_i)_{i=1, \dots, n}$
si vuole costruire una funzione $f(x)$
tale che nei nodi $(x_i)_{i=1, \dots, n}$
soddisfi certe condizioni, dette

Condizioni di interpolazione

Quali sono le condizioni di interpolazione ?



condizioni di interpolazione:
la funzione deve passare per i punti assegnati

Interpolazione di Lagrange: formulazione generale

Sia F uno spazio di funzioni di variabile reale o complessa.

Assegnati:

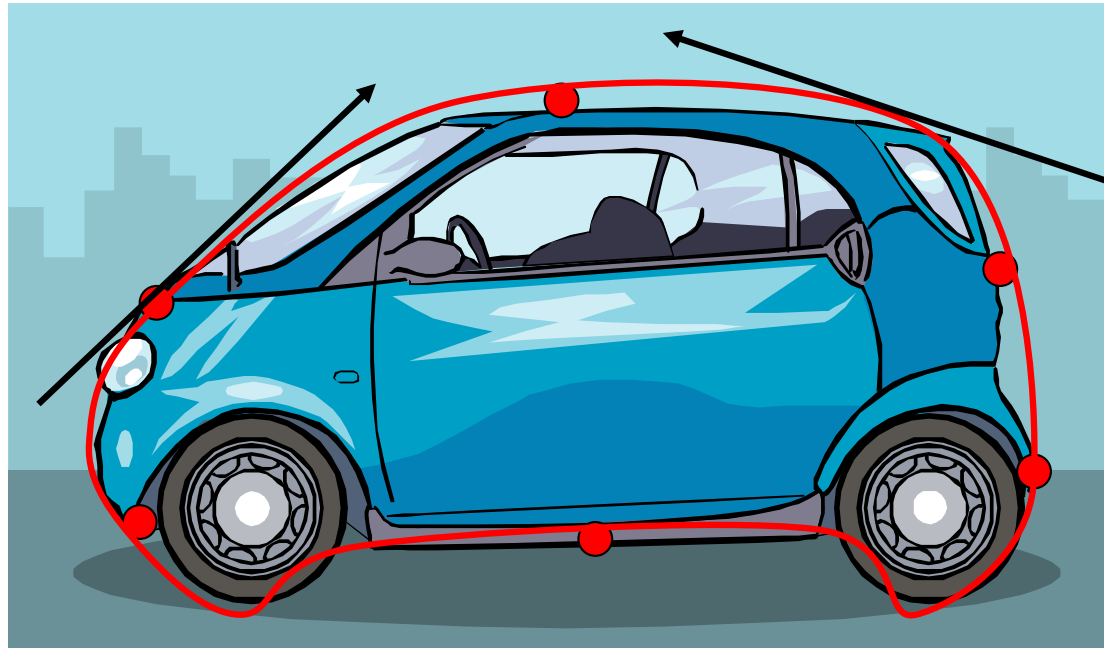
- n valori distinti reali o complessi $\{x_i\}_{i=1,\dots,n}$
- n valori distinti reali o complessi $\{y_i\}_{i=1,\dots,n}$

Si cerca una funzione $f(x) \in F$ tale che:

$$f(x_i) = y_i \quad i=1,\dots,n$$

Condizioni di interpolazione di Lagrange

Quali sono le condizioni di interpolazione ?



condizioni di interpolazione:
la funzione deve passare per i punti assegnati
con una fissata tangente

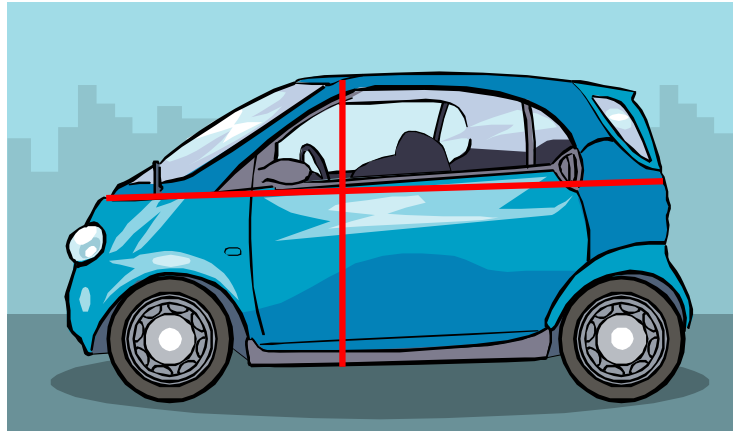
In ciascun nodo sono assegnate condizioni sulla funzione $f(x)$ e sulle sue derivate....

x_1	x_2		x_{n-1}	x_n
y^0_1	y^0_2	...	y^0_{n-1}	y^0_n
y^1_1	y^1_2	...	y^1_{n-1}	y^1_n
	y^2_2	...	y^2_{n-1}	
	y^3_2			

N.B. se è assegnata in un nodo la condizione sulla derivata di ordine q , deve essere assegnata anche la condizione sulle derivate dall'ordine 0 fino a $q-1$.

Interpolazione di Hermite

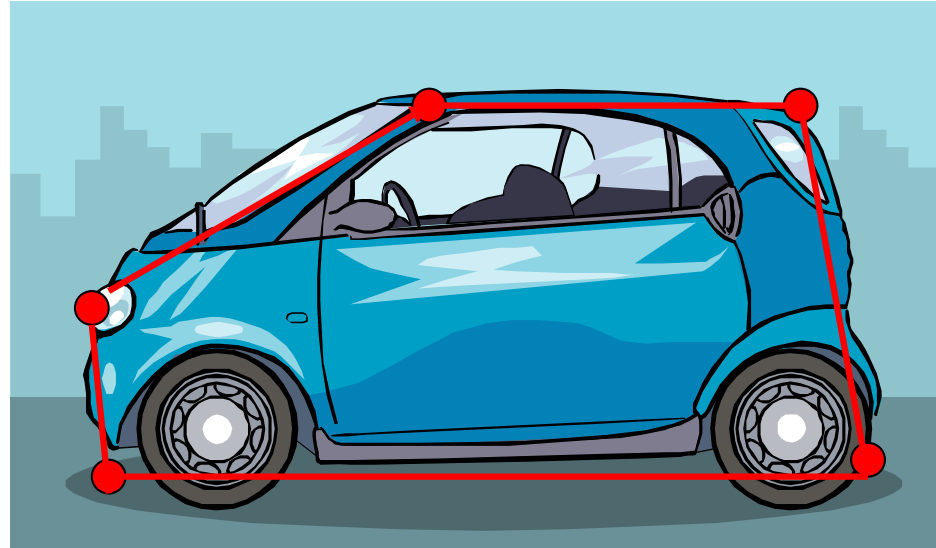
Problema: una volta fissati i punti per i quali deve passare la curva stabiliamo anche dei **vincoli** relativi alla dimensione dell'oggetto:



- lunghezza massima
- larghezza massima
- altezza massima dell'abitacolo
-

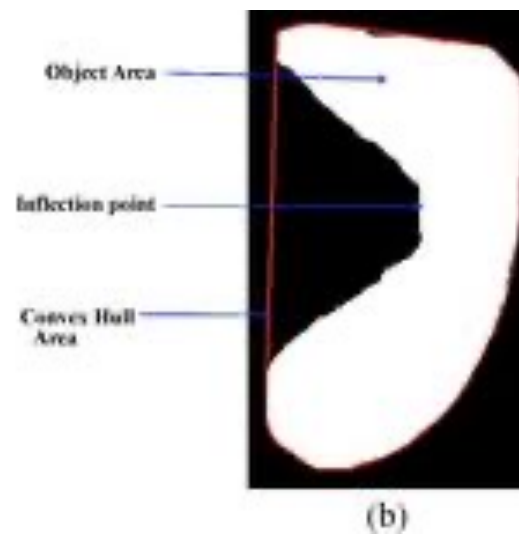
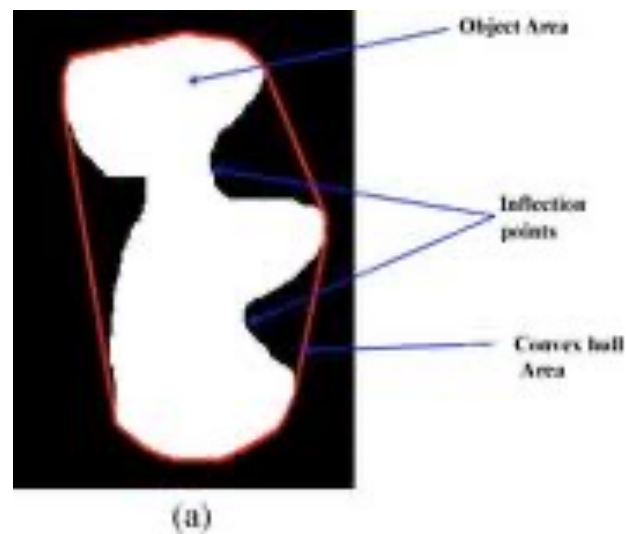
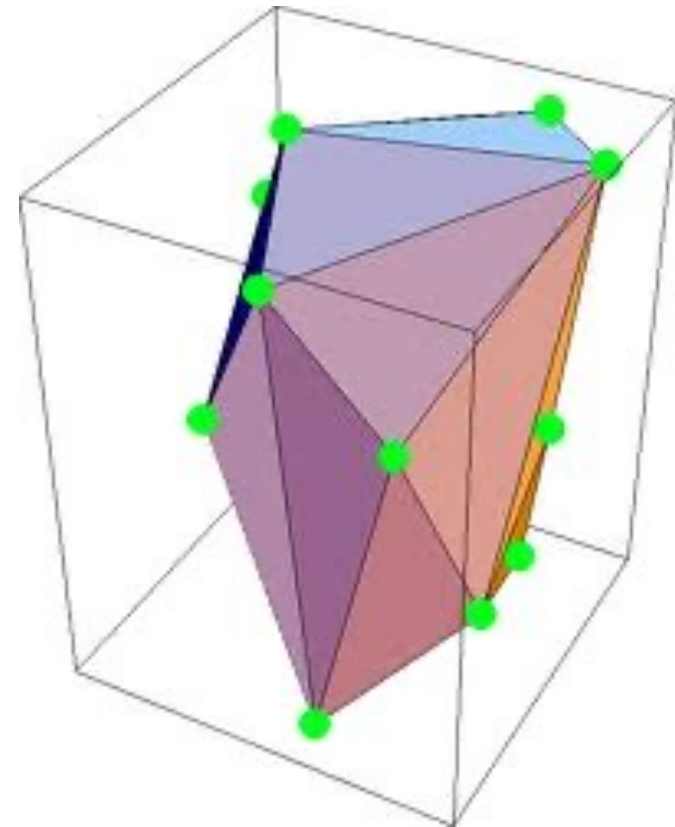
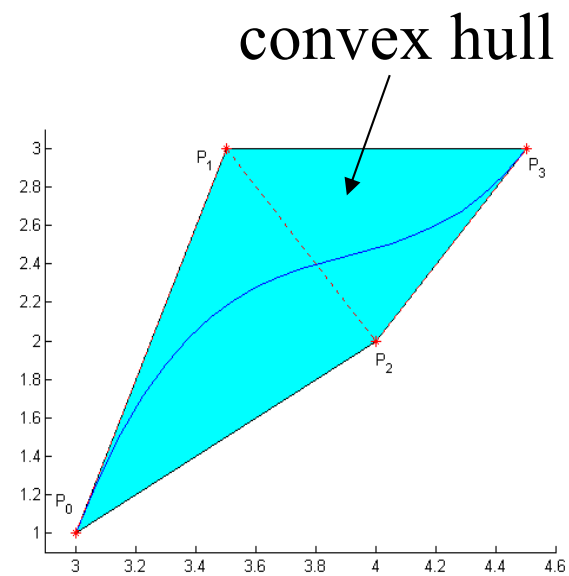
Come si traduce in termini matematici la richiesta che la curva debba tener conto di questi ulteriori vincoli ?

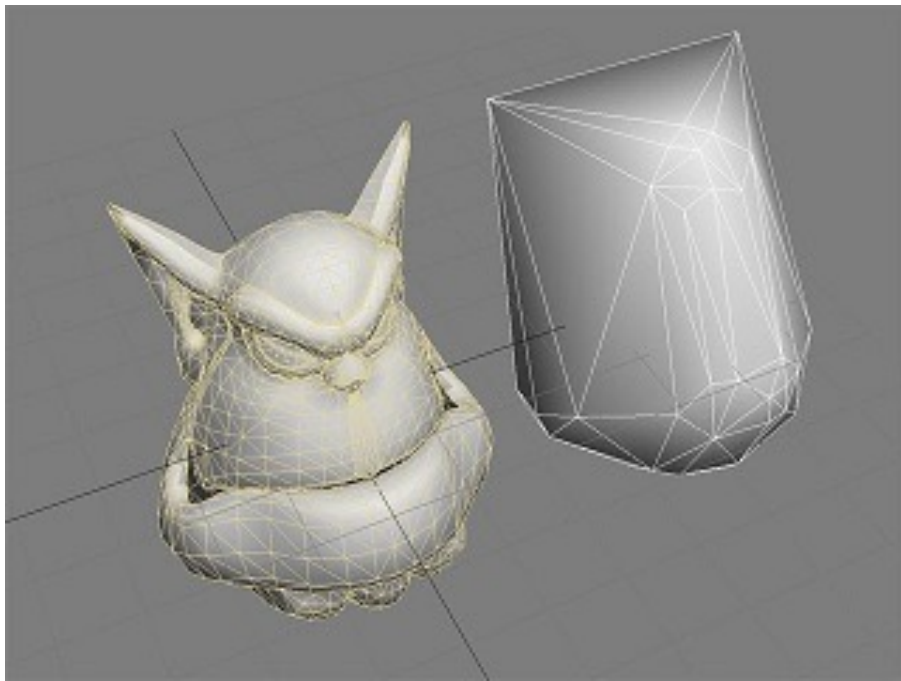
Punti di controllo (1960 - Bezier)



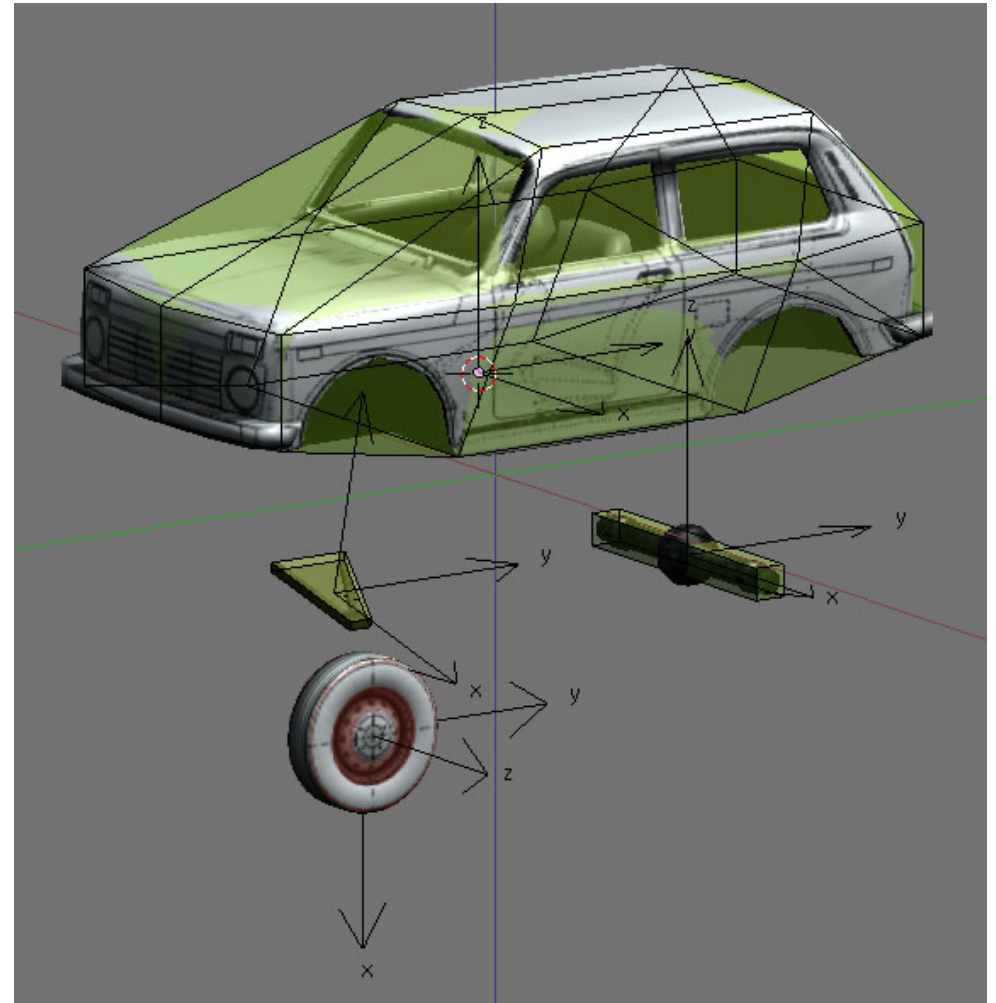
Un modo naturale per stabilire la dimensione dell'oggetto da disegnare
consiste nel fissare alcuni punti
- detti **PUNTI DI CONTROLLO** -
e costruire il più piccolo poligono **convesso** contenente tali punti
(**CONVEX HULL= INVILUPPO CONVESSO**)

Esempi di convex hulls





Esempi di convex hulls



In Computer Graphics

- Tutte le curve sono costruite operando **unicamente** sui punti di controllo (e quindi sul convex hull)
- I punti di controllo sono disposti a partire dai vincoli imposti alla curva

Per tradurre in termini matematici
la costruzione della curva / modello
bisogna stabilirne la forma

I polinomi

Consideriamo i polinomi perché
computazionalmente
vantaggiosi...

Esistono 3 rappresentazioni per un polinomio $p(x)$:

Rappresentazione nella forma Standard

$$p(x) = \sum_{i=0}^{n-1} a_i \varphi_i(x)$$

a_i coefficienti da determinare

x^i



$$p(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$$

Esistono 3 rappresentazioni per il polinomio $p(x)$:

Rappresentazione nella forma di Lagrange

$$p(x) = \sum_{i=1}^n a_i \varphi_i(x)$$

y_i

$$l_i(x) = \prod_{\substack{j=1 \\ i \neq j}}^n \frac{(x - x_j)}{(x_i - x_j)}$$



$$p(x) = y_1 l_1(x) + y_2 l_2(x) + \dots + y_n l_n(x)$$

Esistono 3 rappresentazioni per il polinomio $p(x)$:

Rappresentazione nella forma di Newton

$$p(x) = \sum_{i=0}^{n-1} a_i \varphi_i(x)$$

a_i calcolato a partire
dai primi i nodi
(differenza divisa di ordine $i-1$)

$$\prod_{j=1}^i (x - x_j)$$

$$p(x) = a_0 + a_1(x - x_1) + \dots + a_{n-1}(x - x_1) \dots (x - x_{n-1})$$

Il polinomio interpolante di Lagrange $p \in \Pi_{n-1}$
ha una rappresentazione del tipo:

$$p(x) = \sum_{i=1}^n a_i \varphi_i(x)$$

Diagram illustrating the expansion of the Lagrange polynomial $p(x)$ into its three standard forms:

- Forma Standard: x^{i-1}
- Forma di Lagrange: $l_i(x)$
- Forma di Newton: $\prod_{j=1}^{i-1} (x - x_j)$

Funzioni di base

In generale
in ciascuna rappresentazione
le funzioni di base sono polinomi
che tengono conto –*opportunamente*–
delle condizioni
richieste dal modello

La scelta della base dipende dall'obiettivo per cui si costruisce il polinomio

Quale base conviene usare in Computer Graphics?

Si usa una base di polinomi tutti con lo stesso grado del polinomio che descrive la curva da costruire

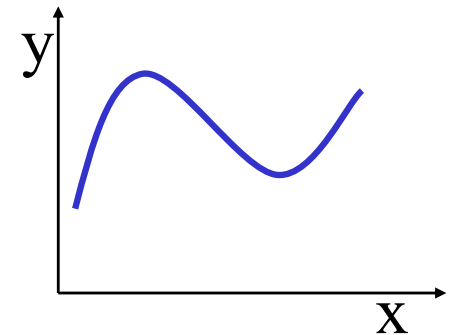
Rappresentazione di curve

- Non parametrica
 - Esplicita
 - Implicita
- Parametrica

Rappresentazione non parametrica esplicita

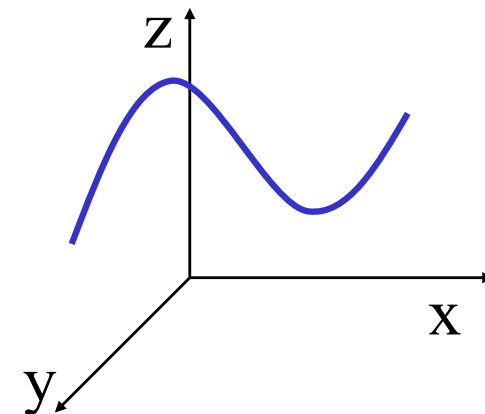
- Curva piana:

$$y = f(x) \quad x \in [a, b]$$



- Curva nello spazio:

$$\begin{cases} y = f(x) \\ z = g(x) \end{cases} \quad x \in [a, b]$$



Rappresentazione non parametrica esplicita

Es. Equazione della retta

$$y = mx + h$$

m=coefficiente angolare
h =intercetta

- Il calcolo di punti sulla curva è semplice
- Fornisce un unico valore della variabile dipendente per ciascun valore della variabile indipendente
 - Non consente di rappresentare curve chiuse o a più valori (es. circonferenza, rette verticali, ...)

Rappresentazione non parametrica implicita

- Curva piana:

$$f(x, y) = 0$$

- Curva nello spazio:

$$\begin{cases} f(x, y, z) = 0 \\ g(x, y, z) = 0 \end{cases}$$

intersezione (se esiste) di due superfici anch'esse rappresentate in forma implicita

Rappresentazione non parametrica implicita

Es. Equazione della retta

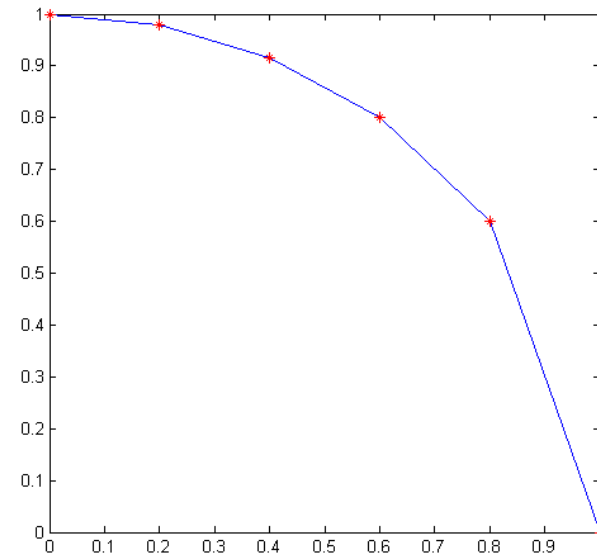
$$ax + by + c = 0$$

- Consente di rappresentare tutte le curve
 - Es. Circonferenza: $x^2 + y^2 - r^2 = 0$
- Il calcolo di punti sulla curva è in genere difficile

Rappresentazione non parametrica

- È dipendente dal sistema di riferimento
 - La scelta del sistema di riferimento può influenzare la facilità nell'uso delle curve
- Quando i punti sono calcolati con uguali incrementi della variabile indipendente, questi non sono distribuiti uniformemente lungo la curva
 - Influenza sull'accuratezza e sulla qualità dell'output grafico

$$y = \sqrt{1 - x^2} \quad x \in [0, 1]$$

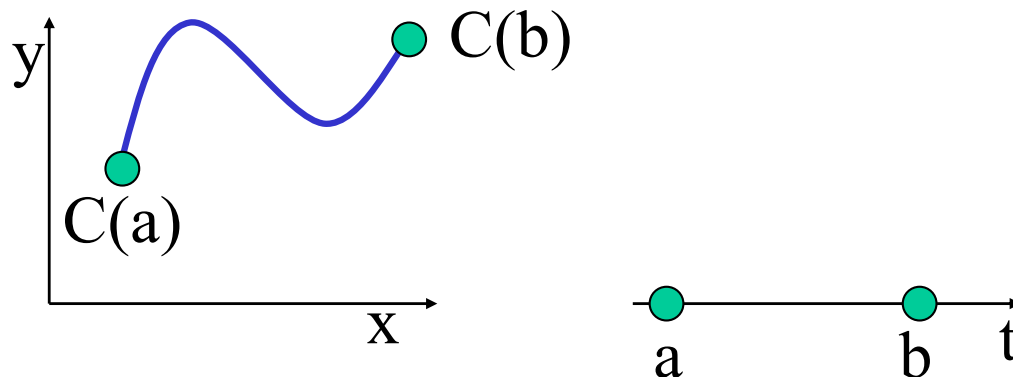


Rappresentazione parametrica

- Curva piana:

$$\begin{cases} x = x(t) \\ y = y(t) \end{cases} \quad C(t) = (x(t), y(t)) \quad t \in [a, b]$$

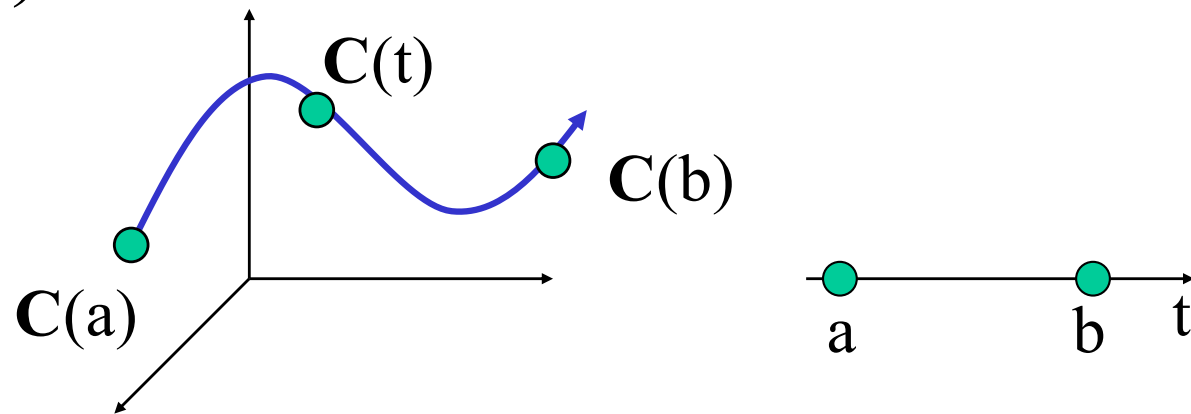
- Al variare di t in $[a, b]$, le coordinate $(x(t), y(t))$ individuano un punto che si sposta sulla curva



Rappresentazione parametrica

- Curva nello spazio:

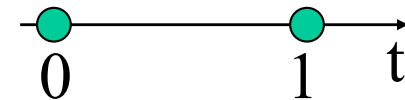
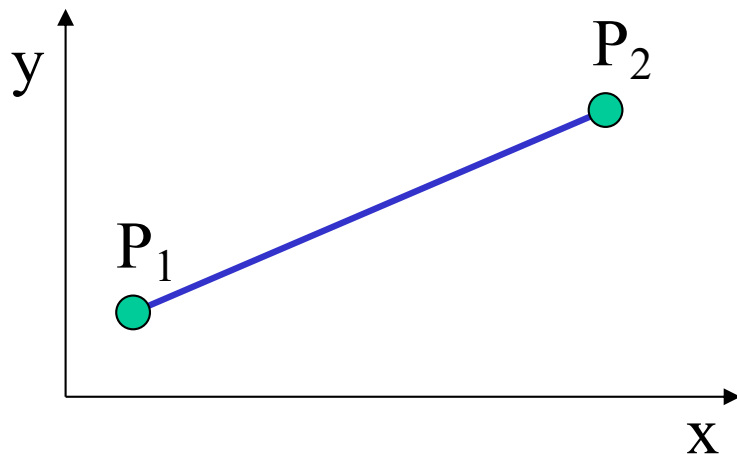
$$\begin{cases} x = x(t) \\ y = y(t) \\ z = z(t) \end{cases} \quad C(t) = (x(t), y(t), z(t)) \quad t \in [a, b]$$



Rappresentazione parametrica: esempio

Retta di estremi $P_1(x_1, y_1)$ e $P_2(x_2, y_2)$:

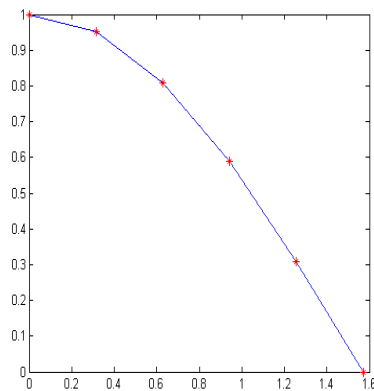
$$\begin{cases} x = x_1 + t * (x_2 - x_1) \\ y = y_1 + t * (y_2 - y_1) \end{cases} \quad t \in [0,1]$$



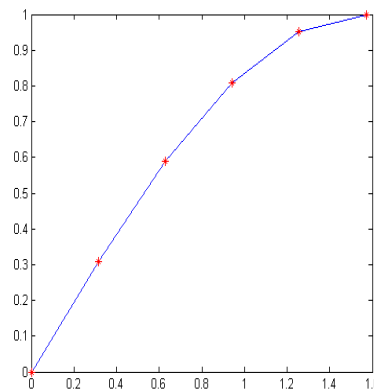
Rappresentazione parametrica: esempio

Arco di circonferenza di centro (0,0) e raggio 1:

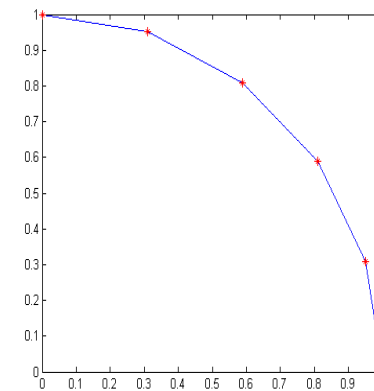
$$\begin{cases} x(t) = \cos(t) \\ y(t) = \sin(t) \end{cases} \quad t \in \left[0, \frac{\pi}{2}\right]$$



$$x = \cos(t)$$



$$y = \sin(t)$$

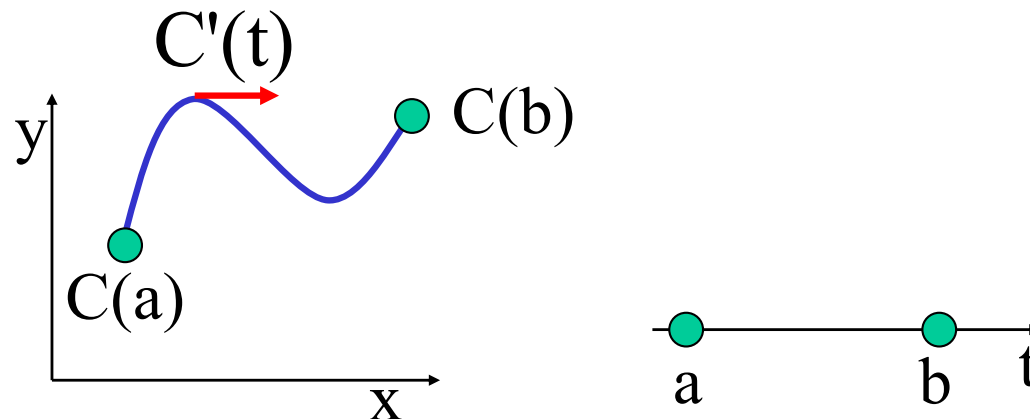


$$C(t) = (\cos(t), \sin(t))$$

Rappresentazione parametrica

Modello fisico: un punto che si muove nel tempo

- Ad ogni istante t , la posizione del punto è data da $C(t)=(x(t),y(t),z(t))$
- La velocità del punto sulla curva è data dalla derivata rispetto al tempo t : $C'(t)=(x'(t),y'(t),z'(t))$



Rappresentazione parametrica

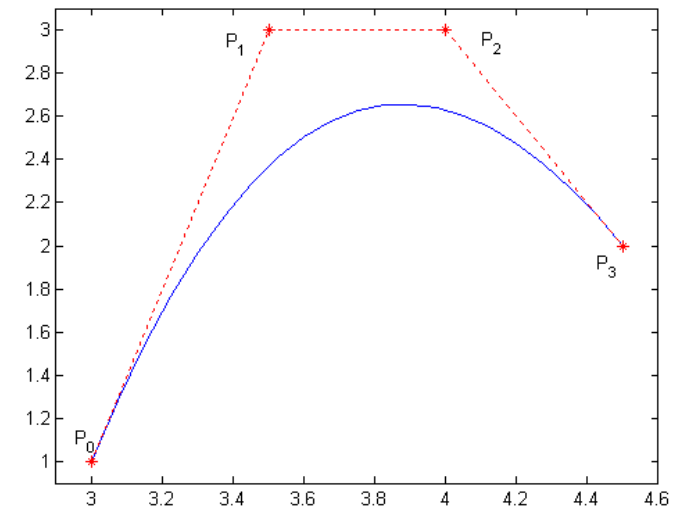
- È esplicita → Il calcolo di punti della curva è semplice
- Consente di rappresentare curve chiuse o con valori multipli
- Adatta alla formulazione di curve composite (unione di più curve)
- Consente di esprimere una curva come combinazione lineare di funzioni scalari dei parametri, con coefficienti vettoriali:

$$C(t) = \sum_{i=0}^n \mathbf{c}_i f_i(t) \quad t \in [a, b]$$

- Una qualunque trasformazione geometrica può essere ottenuta applicando la trasformazione al vettore dei coefficienti e lasciando inalterata la formulazione matematica

Curve di Bezier

- $P_0, \dots, P_n$ *punti di controllo*



$$C(t) = \sum_{i=0}^n P_i B_{i,n}(t) \quad t \in [0,1]$$

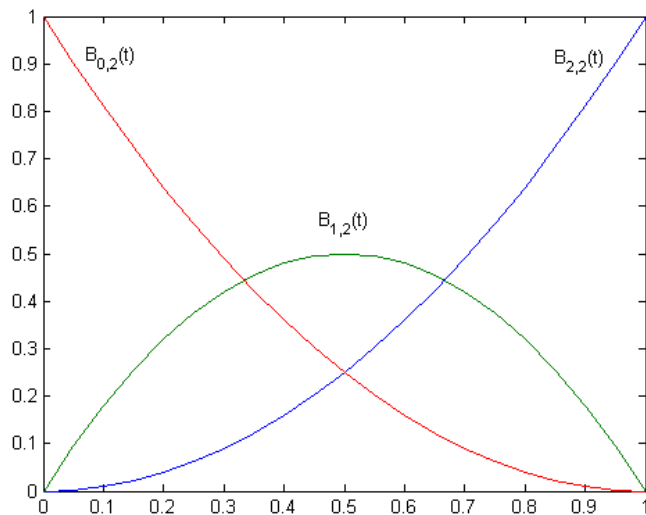
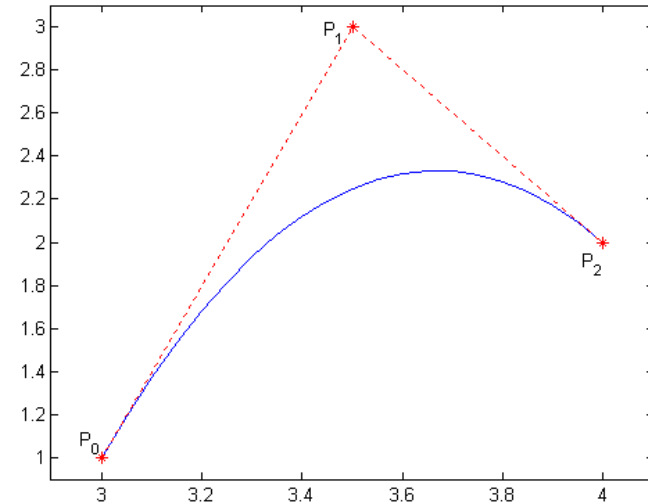
con $B_{i,n}(t)$ polinomi di Bernstein di grado n

$$B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i} \quad t \in [0,1]$$

Coefficiente binomiale $\longrightarrow \binom{n}{i} = \frac{n!}{i!(n-i)!}$

Esempio: curva di Bezier di grado 2

$$C(t) = P_0 B_{0,2}(t) + P_1 B_{1,2}(t) + P_2 B_{2,2}(t)$$
$$t \in [0,1]$$



Polinomi di Bernstein di grado 2:

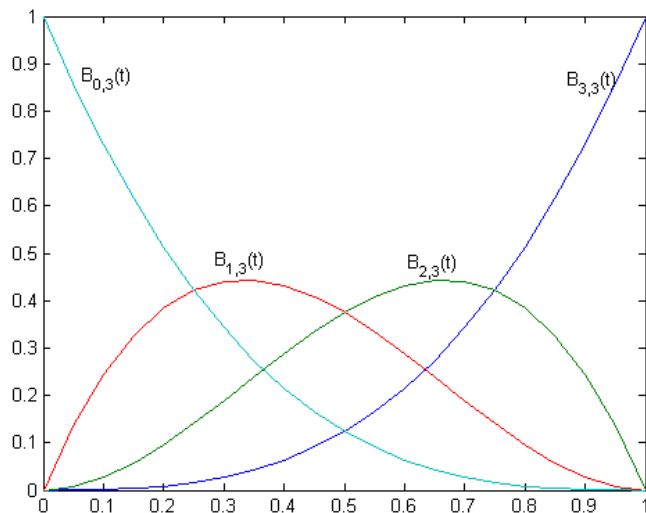
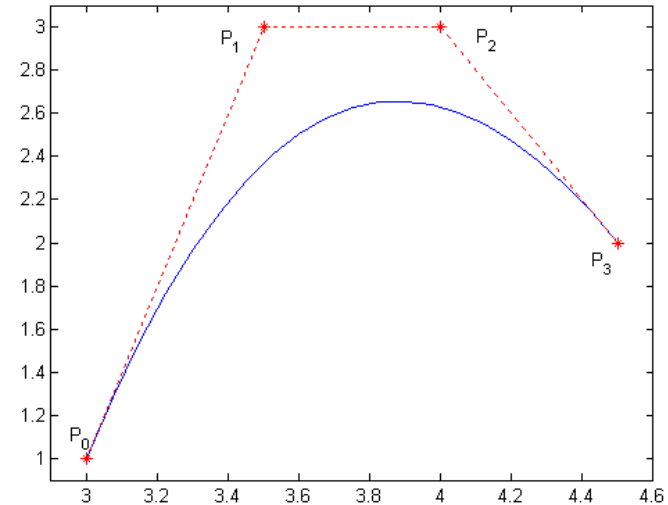
$$B_{2,2}(t) = t^2$$

$$B_{1,2}(t) = 2t(1-t)$$

$$B_{0,2}(t) = (1-t)^2$$

Esempio: curva di Bezier di grado 3

$$C(t) = P_0 B_{0,3}(t) + P_1 B_{1,3}(t) + P_2 B_{2,3}(t) + P_3 B_{3,3}(t)$$
$$t \in [0,1]$$



Polinomi di Bernstein di grado 3:

$$B_{3,3}(t) = t^3$$

$$B_{2,3}(t) = 3t^2(1-t)$$

$$B_{1,3}(t) = 3t(1-t)^2$$

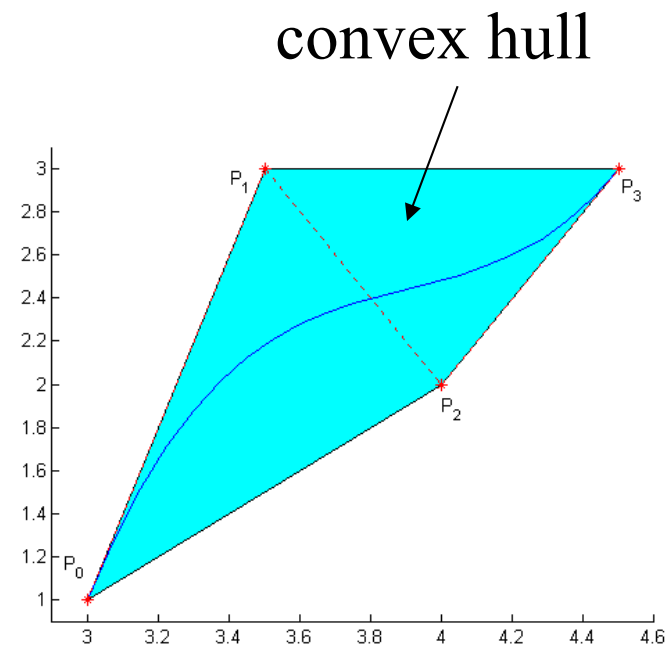
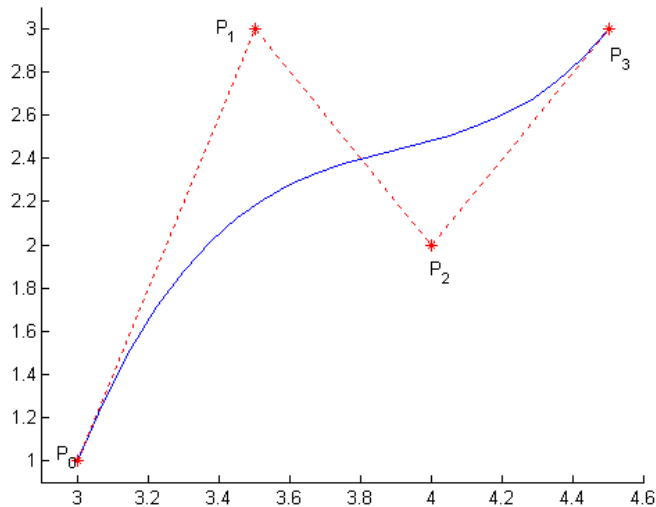
$$B_{0,3}(t) = (1-t)^3$$

Proprietà dei polinomi di Bernstein

- *Non negatività:* $B_{i,n}(t) \geq 0 \quad \forall t \in [0,1]$
- *Partizione dell'unità:* $\sum_{i=0}^n B_{i,n}(t) = 1 \quad \forall t \in [0,1]$
- $\{B_{0,n}(t), \dots, B_{n,n}(t)\}$ *base* per lo spazio dei polinomi di grado al più n

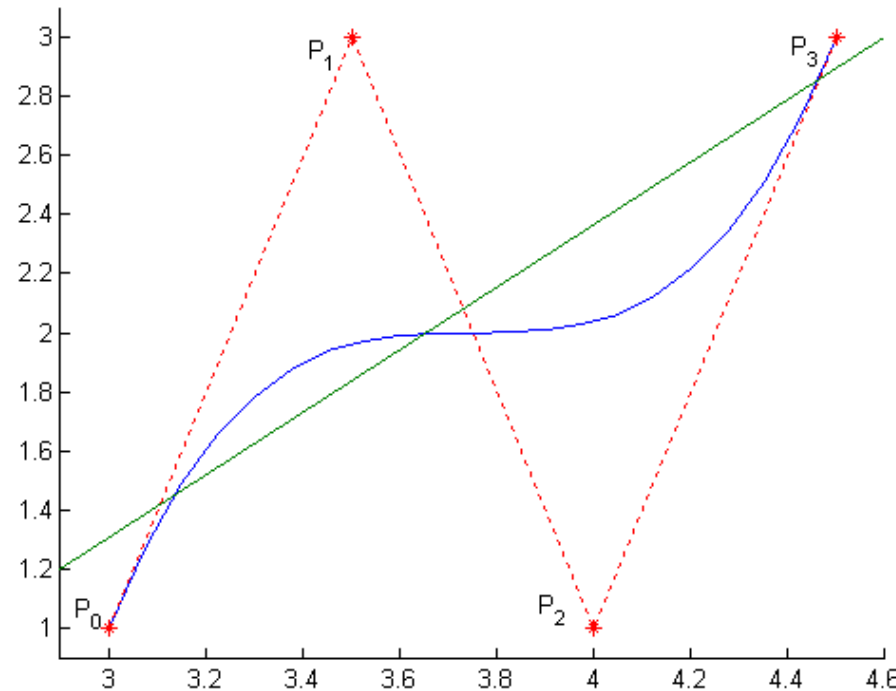
Proprietà delle curve di Bezier

- *Interpolazione*: $C(t)$ interpola P_0 e P_n
- *Convex hull*: giace completamente all'interno del più piccolo insieme convesso contenente i punti di controllo



Proprietà delle curve di Bezier (cont.)

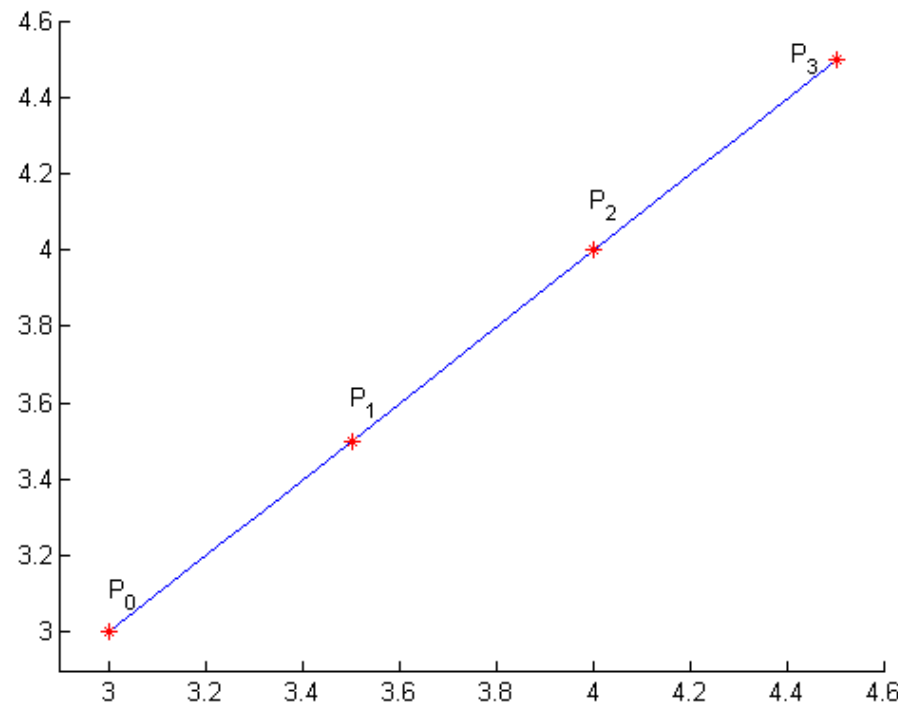
- *Variation diminishing*: nessuna retta (o piano) ha più intersezioni con la curva che con il poligono di controllo



La curva non *varia* più del suo poligono di controllo

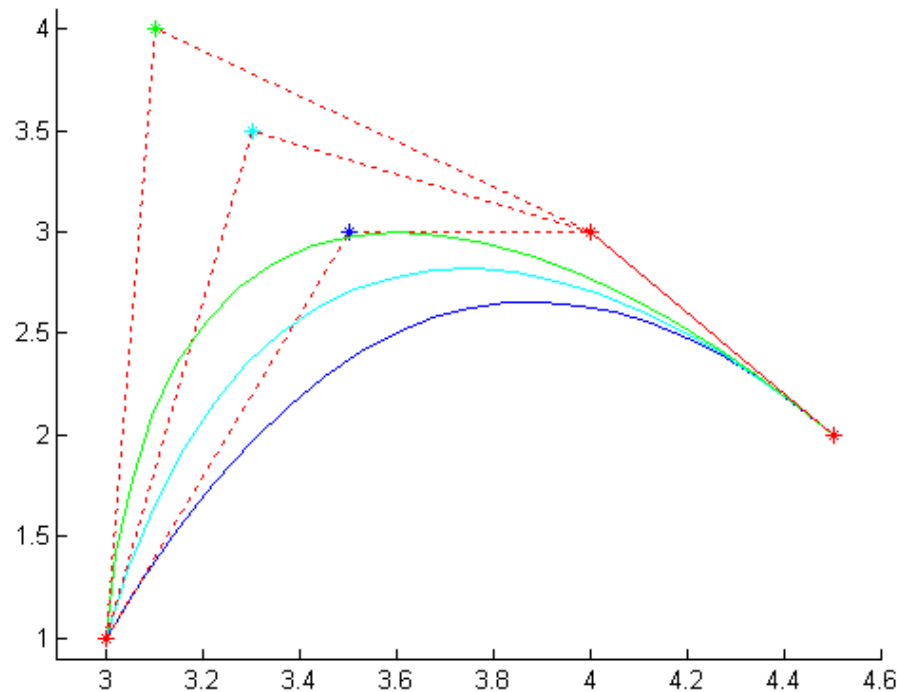
Proprietà delle curve di Bezier (cont.)

- *Precisione lineare*: se tutti i punti di controllo sono allineati, la curva è una retta passante per i punti stessi



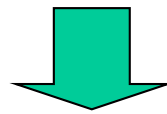
Proprietà delle curve di Bezier (cont.)

- *Controllo globale*: spostando un punto di controllo la forma della curva viene interamente modificata



Proprietà delle curve di Bezier (cont.)

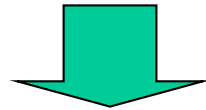
- *Interpolazione*
- *Convex hull*
- *Variation diminishing*
- *Precisione lineare*
- *Controllo globale*



Chiaro significato geometrico dei punti di controllo

Proprietà delle curve di Bezier (cont.)

- *Invarianza affine*: la relazione fra la curva e i punti di controllo è invariante per trasformazioni affini



Per applicare una trasformazione affine ad una curva di Bezier è sufficiente applicarla ai soli punti di controllo

Proprietà delle curve di Bezier (cont.)

- Una curva di Bezier è *continua* con tutte le sue derivate
- La *derivata prima* di una curva di Bezier è una curva di Bezier di grado $n-1$ ottenuta differenziando il poligono di controllo originale:

$$C'(t) = \frac{d}{dt} C(t) = n \sum_{i=0}^{n-1} (P_{i+1} - P_i) B_{i,n-1}(t) \quad t \in [0,1]$$

In particolare:

$$C'(0) = n(P_1 - P_0) \qquad C'(1) = n(P_n - P_{n-1})$$



- Una curva di Bezier è tangente al primo e all'ultimo segmento del poligono di controllo

Algoritmo per la valutazione della curva di Bezier

- Dati di input :
 - $n+1$ punti di controllo
 - Valore di t in cui si vuole valutare la curva
- Dati di output
 - punto sulla curva

Algoritmo di de Casteljau (1959, Renault)

Idea di base (n=2)

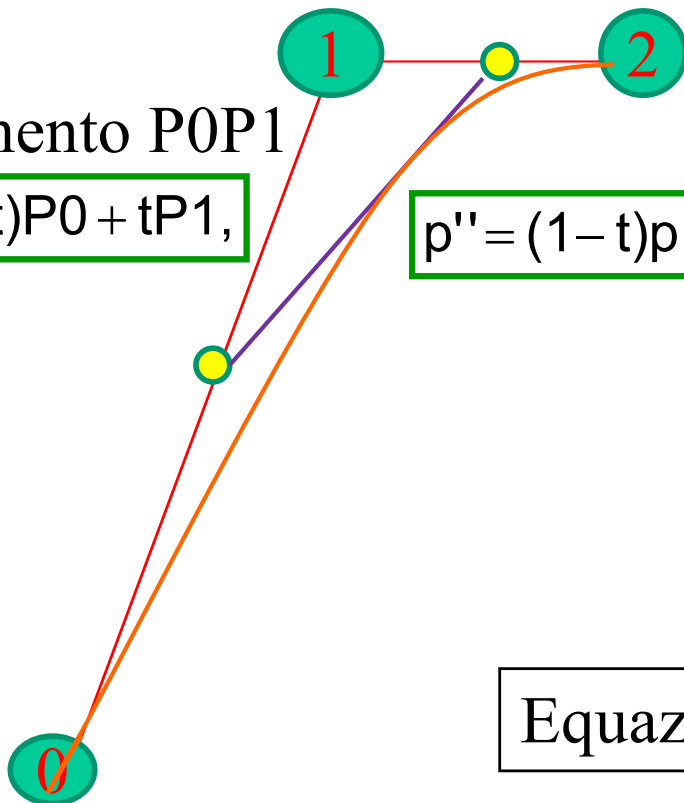
Segmento P1P2

$$p' = (1-t)P1 + tP2,$$

Segmento P0P1

$$p = (1-t)P0 + tP1,$$

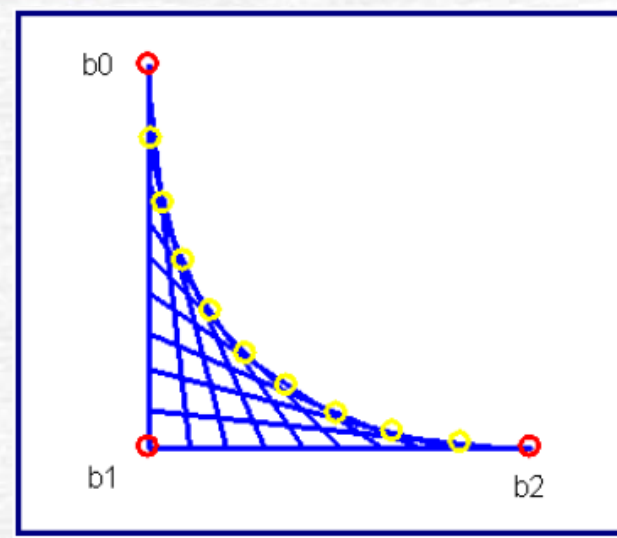
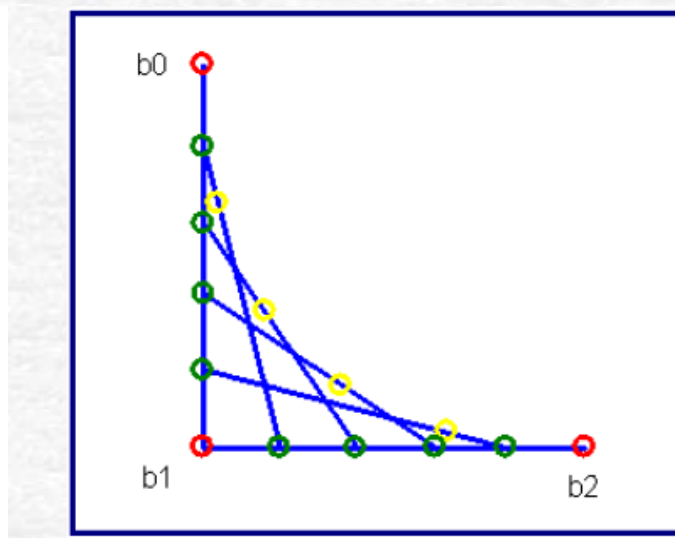
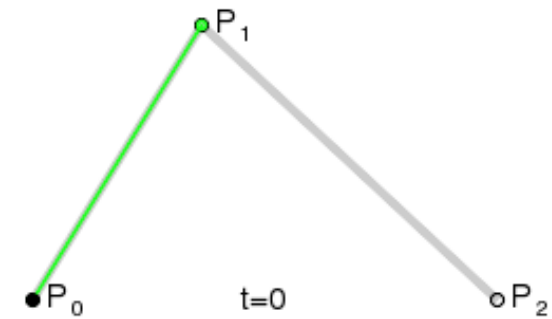
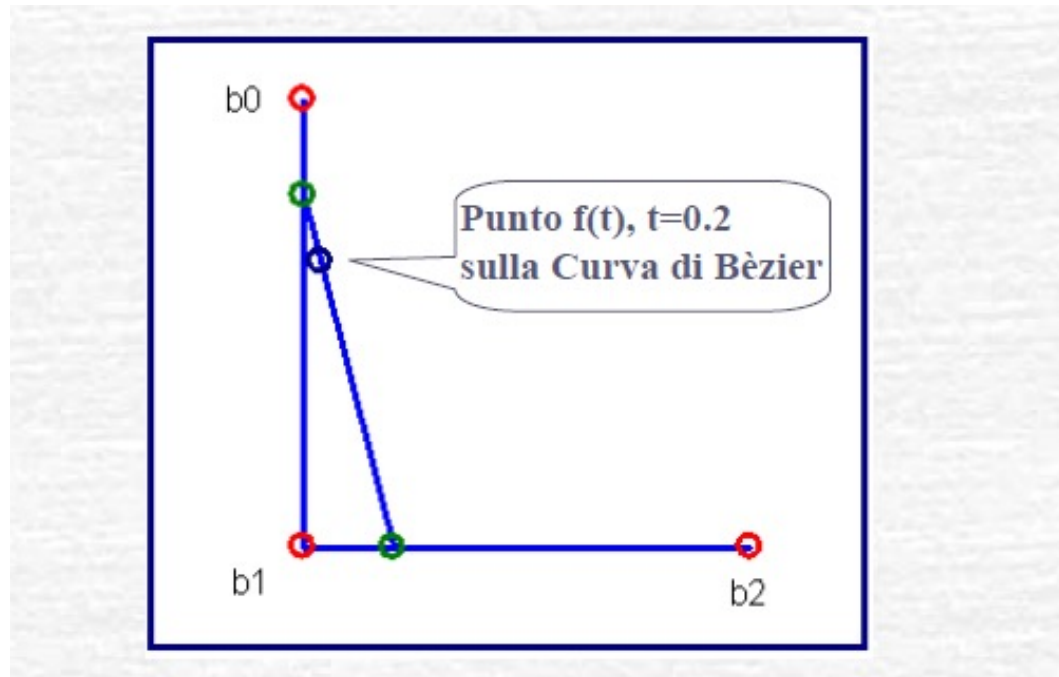
$$p'' = (1-t)p + tp',$$



Equazione della curva di Bezier di grado 2

$$p'' = (1-t)[(1-t)P0 + tP1] + t[(1-t)P1 + tP2] = (1-t)^2P0 + 2t(1-t)P1 + t^2P2$$

Perché p'' è un punto della curva ?

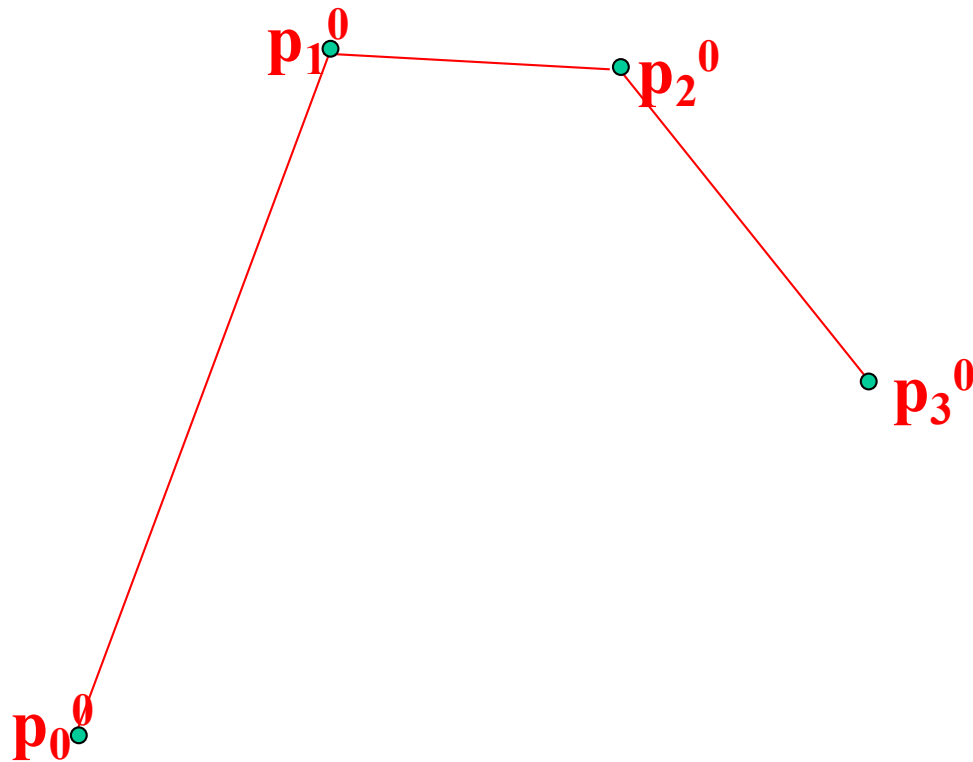


Perché p'' è un punto della curva ?

- p'' appartiene ad una **retta tangente** alla curva di Bezier interna al poligono di controllo
- Al variare di t i punti del tipo p'' descrivono la curva
- La rappresentazione è tanto più affidabile quanto più si infittisce la maglia così ottenuta (rettificazione di una curva)

algoritmo di de Casteljau $n=3$

Passo 0



P_0
 p_0^0

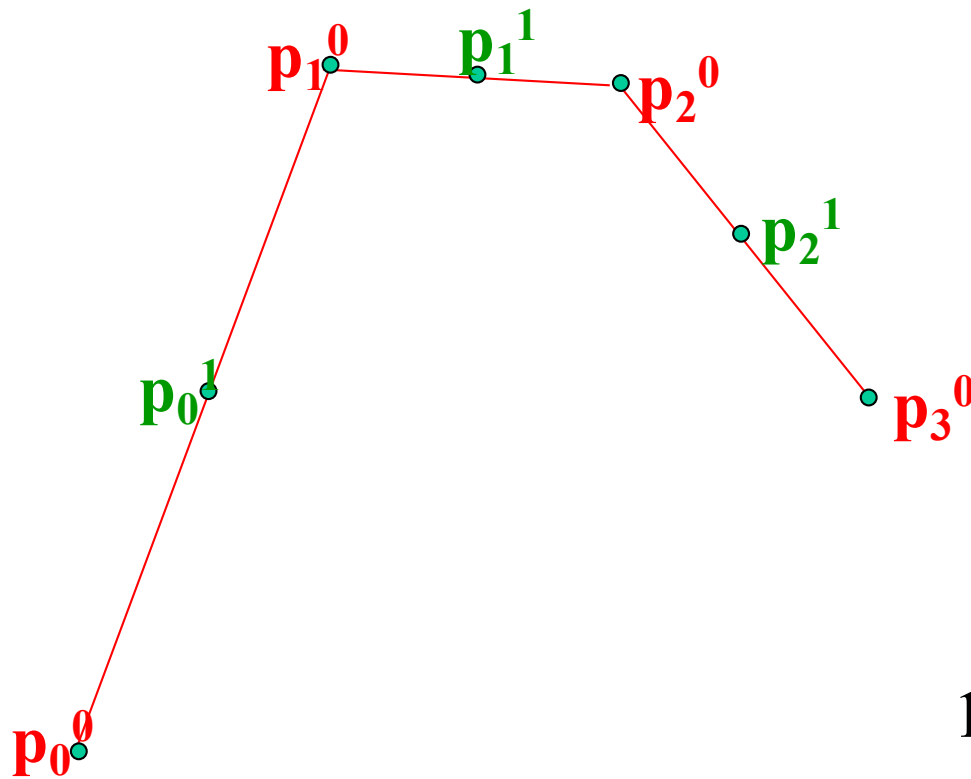
P_1
 p_0^1

P_2
 p_0^2

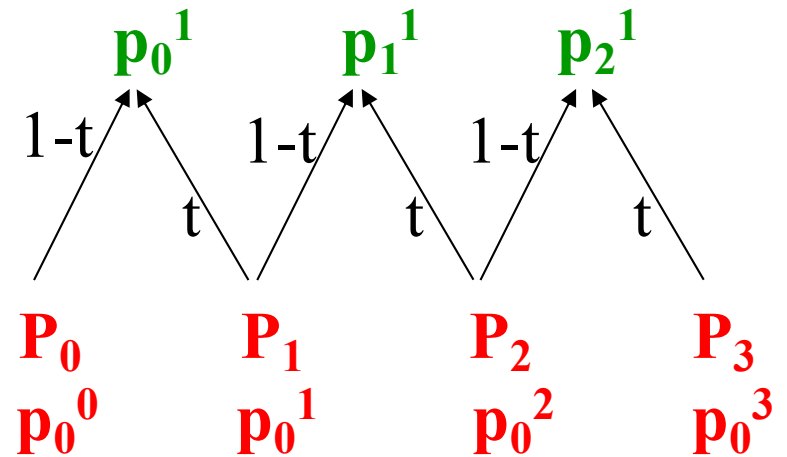
P_3
 p_0^3

algoritmo di de Casteljau n=3

Passo 1

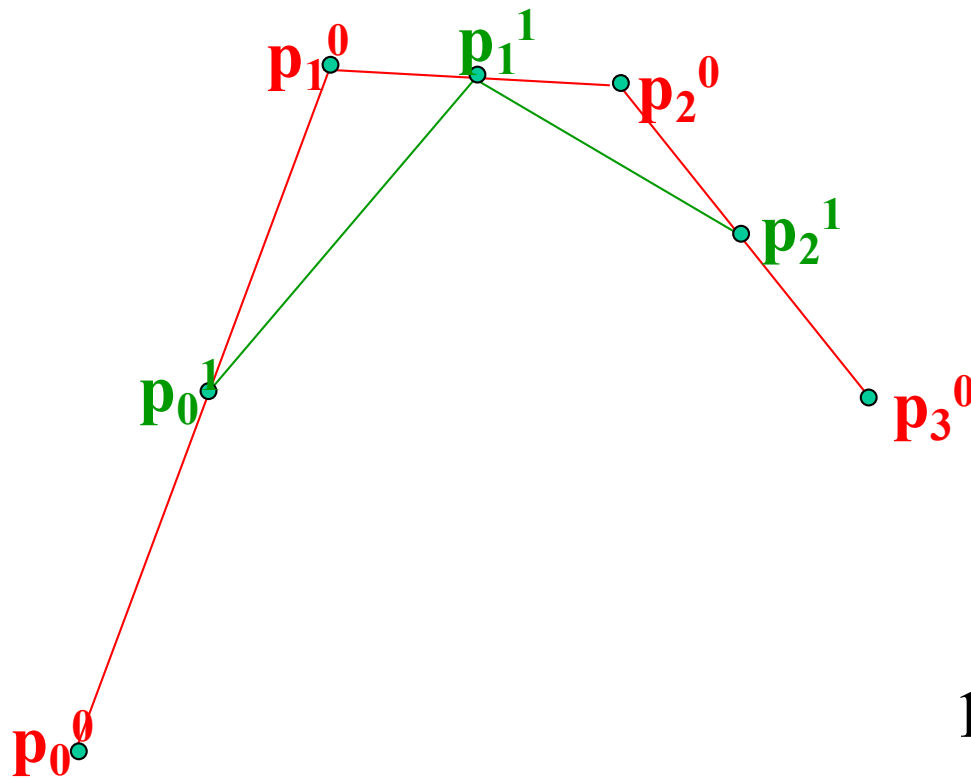


$$p_i^1 = (1-t)p_i^0 + tp_{i+1}^0, \quad i = 0, \dots, 2$$

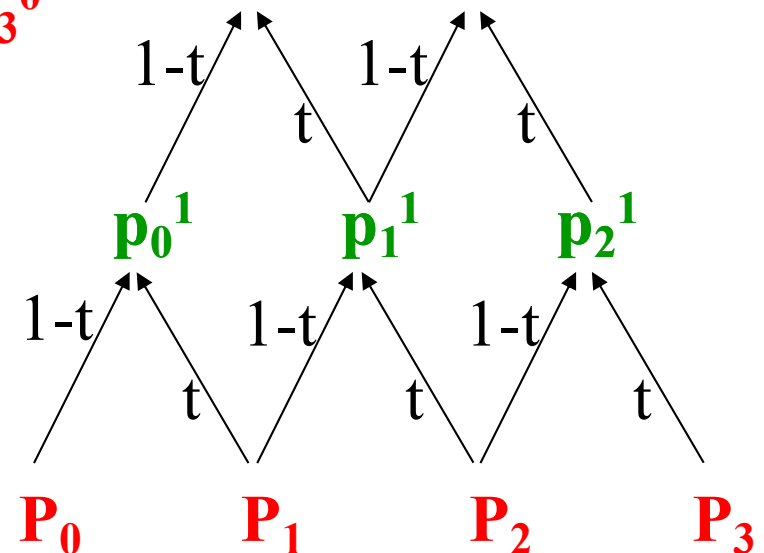


algoritmo di de Casteljau n=3

Passo 1

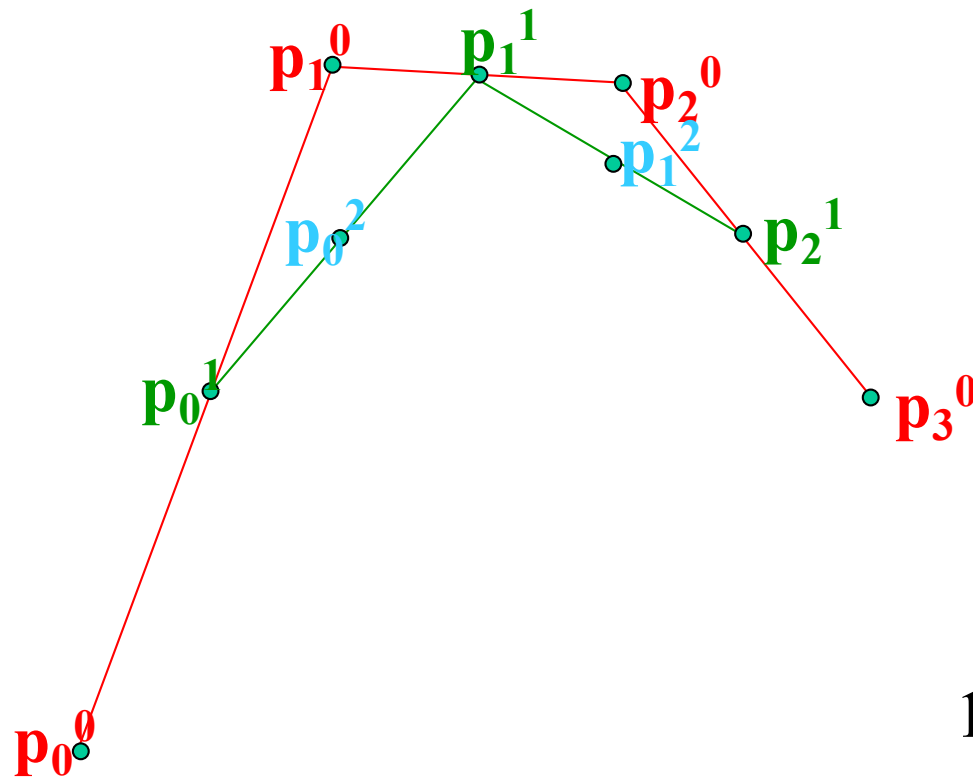


$$p_i^1 = (1-t)p_i^0 + tp_{i+1}^0, \quad i = 0, \dots, 2$$

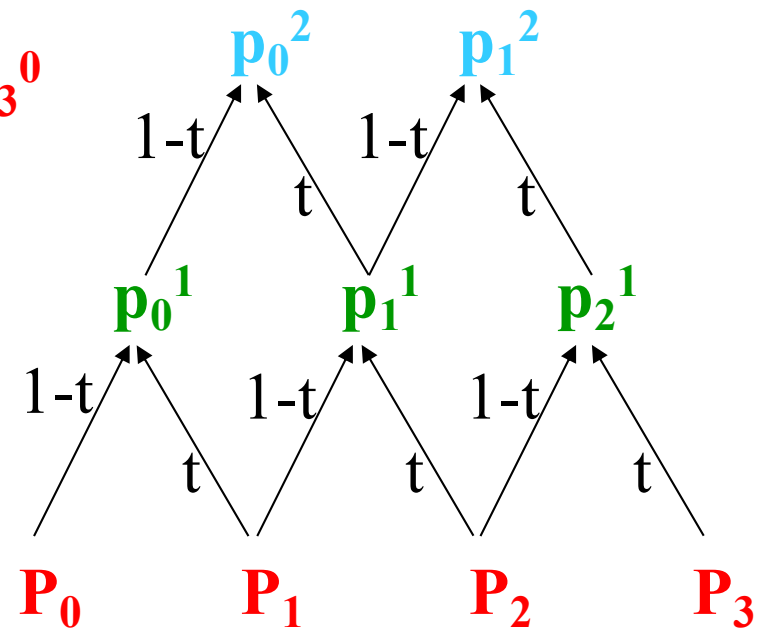


Es. algoritmo di de Casteljau n=3

Passo 2

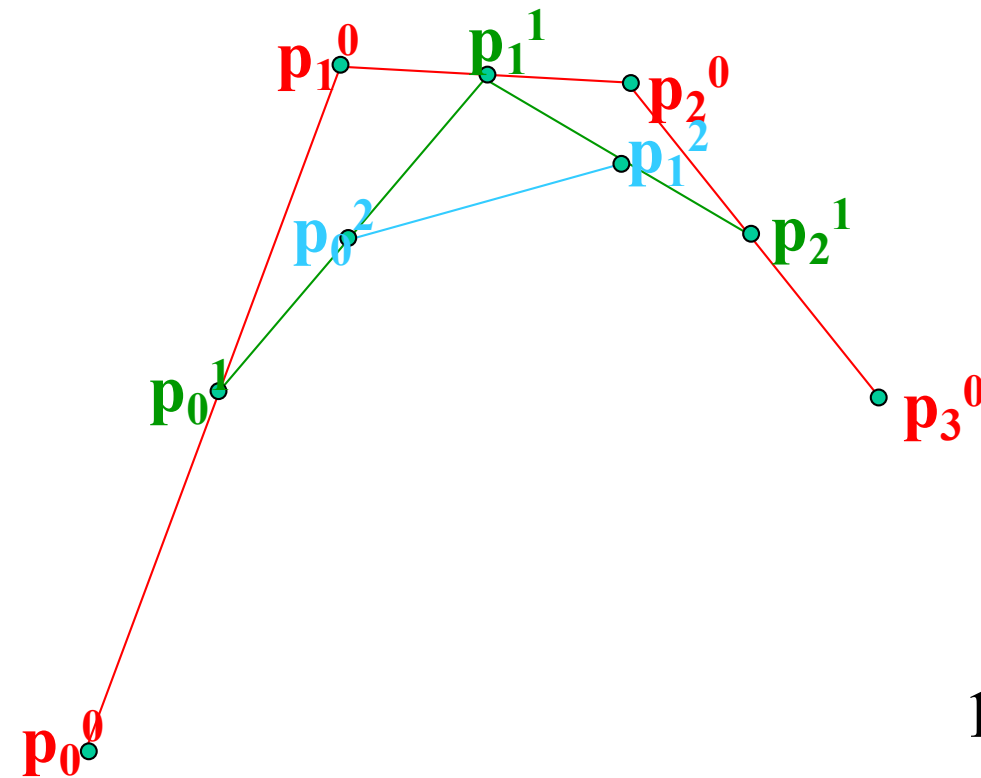


$$p_i^2 = (1-t)p_i^1 + tp_{i+1}^1, \quad i = 0,1$$

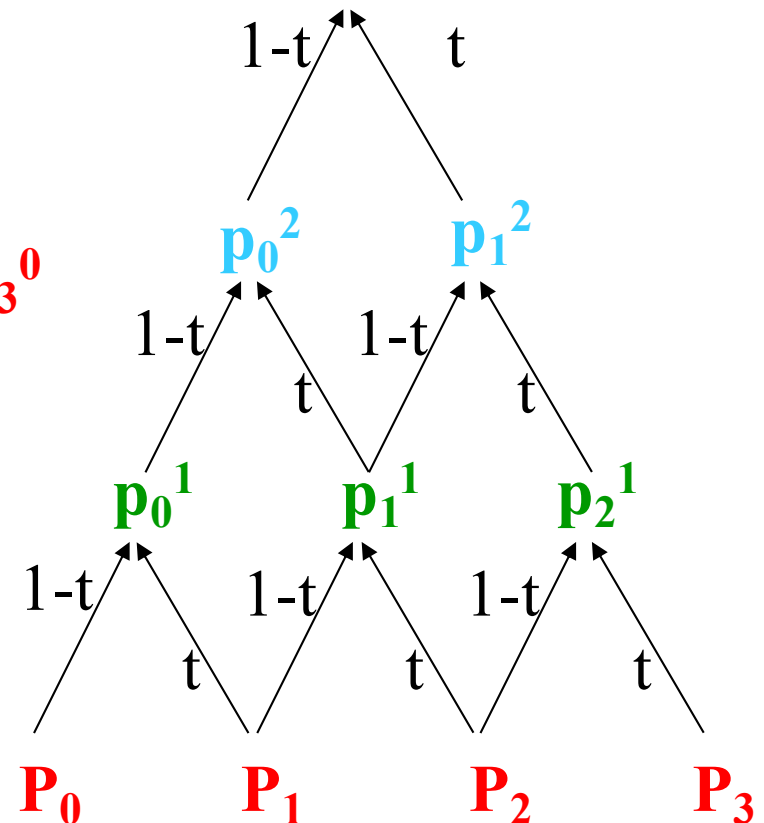


Es. algoritmo di de Casteljau n=3

Passo 2

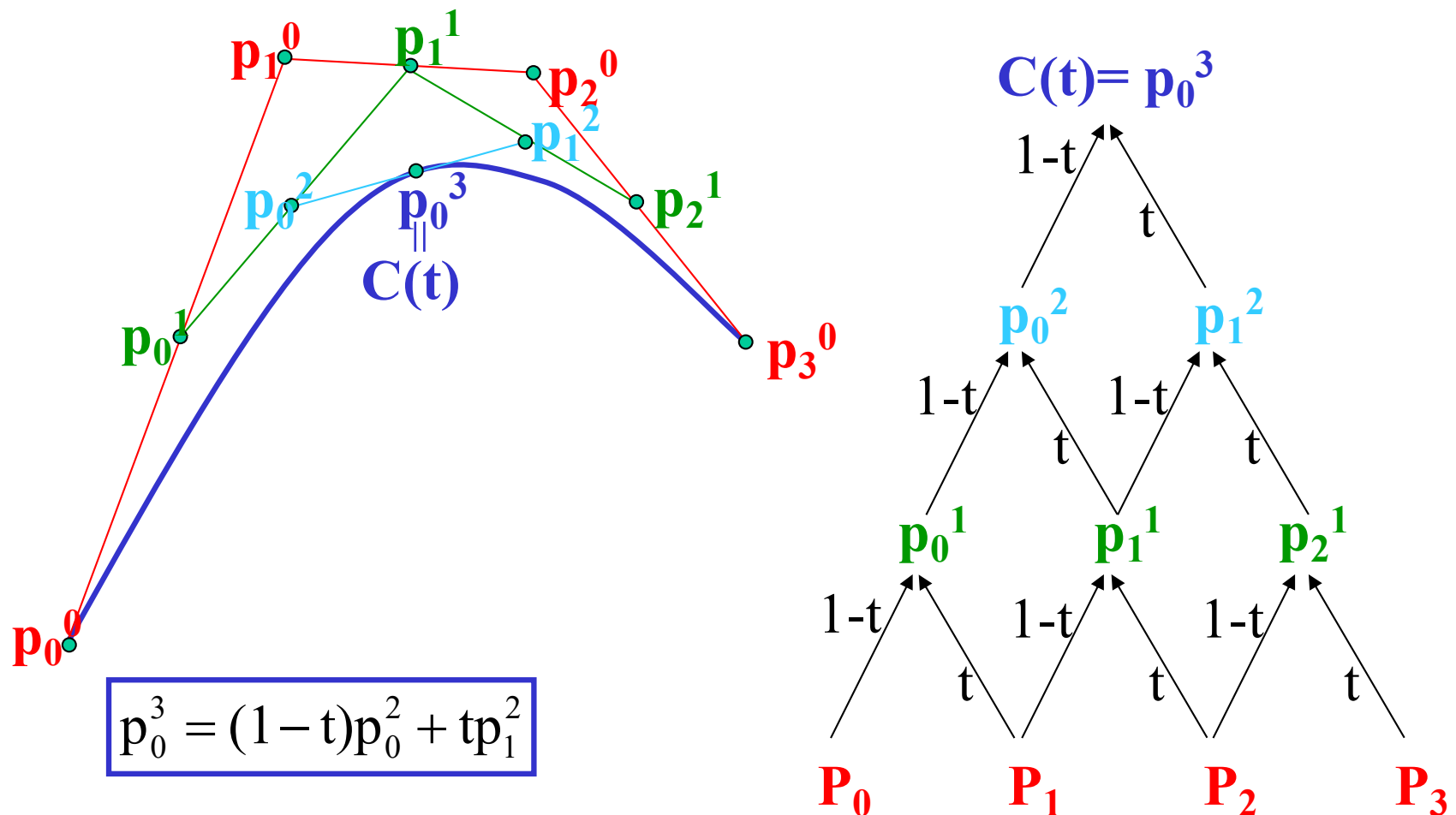


$$p_i^2 = (1-t)p_i^1 + tp_{i+1}^1, \quad i = 0, 1$$



Es. algoritmo di de Casteljau n=3

Passo 3



Valutazione di una curva di Bezier

Algoritmo di de Casteljau

Assegnati: - i punti di controllo $P_0, \dots, P_n$
- un valore del parametro t in $[0,1]$

for $i=0, n$

$p_i^0 = P_i$

endfor

for $r=1, n$

for $i=0, n-r$

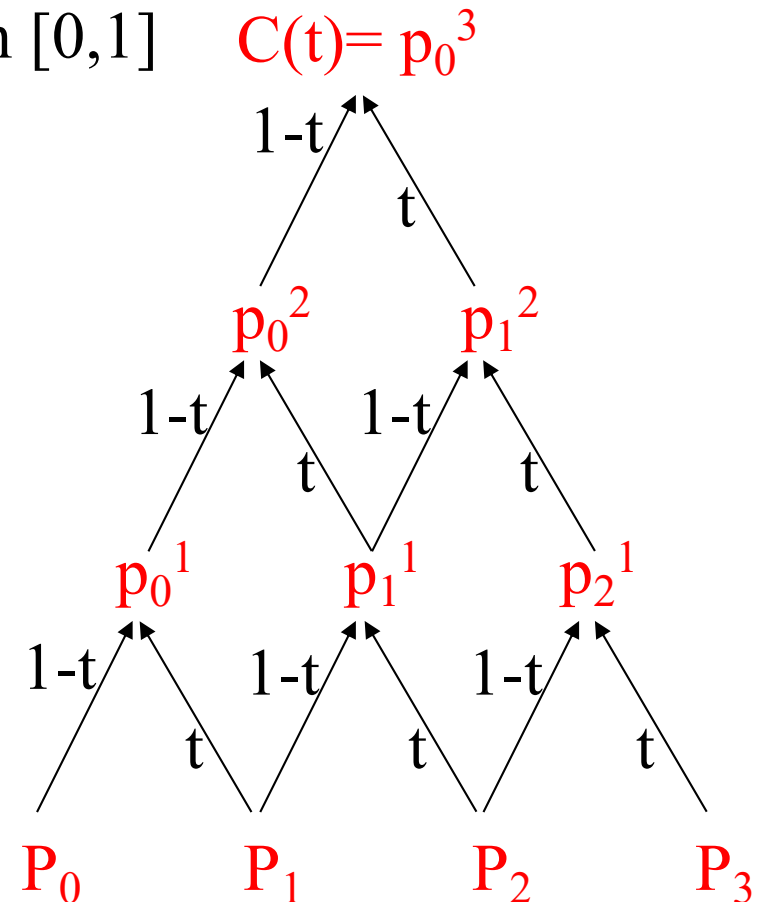
$p_i^r = (1-t) p_i^{r-1} + t p_{i+1}^{r-1}$

endfor

endfor

Risultato: $C(t) = p_0^n$

$T_{DC}(n) = O(n^2)$

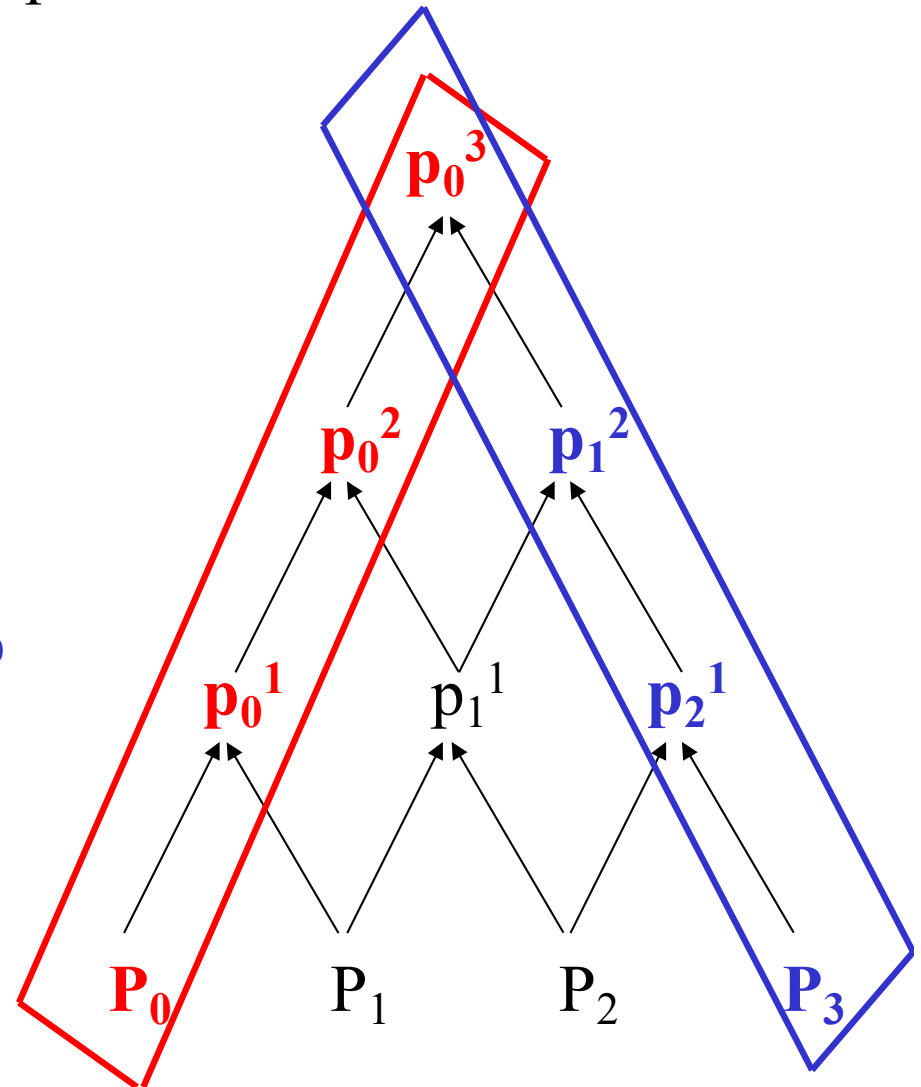
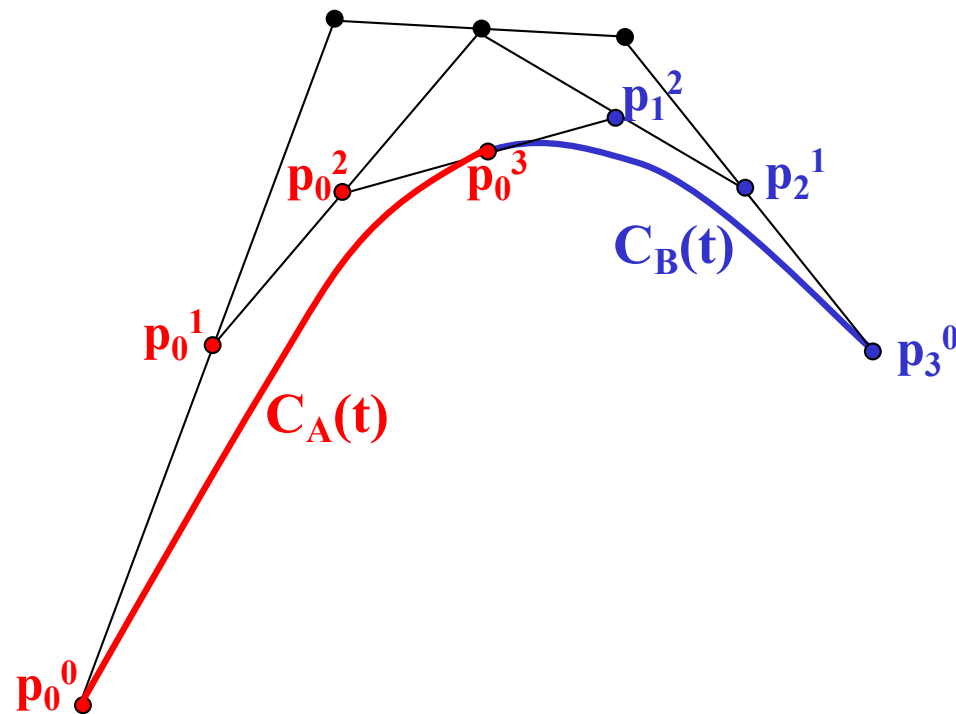


Suddivisione di curve di Bezier

L'algoritmo di de Casteljau è usato per suddividere una curva di Bezier in 2 segmenti di curva

$C_A(t)$ con p.ti $p_0^0, p_0^1, p_0^2, p_0^3$

$C_B(t)$ con p.ti $p_0^3, p_1^2, p_2^1, p_3^0$



Rendering di curve di Bezier

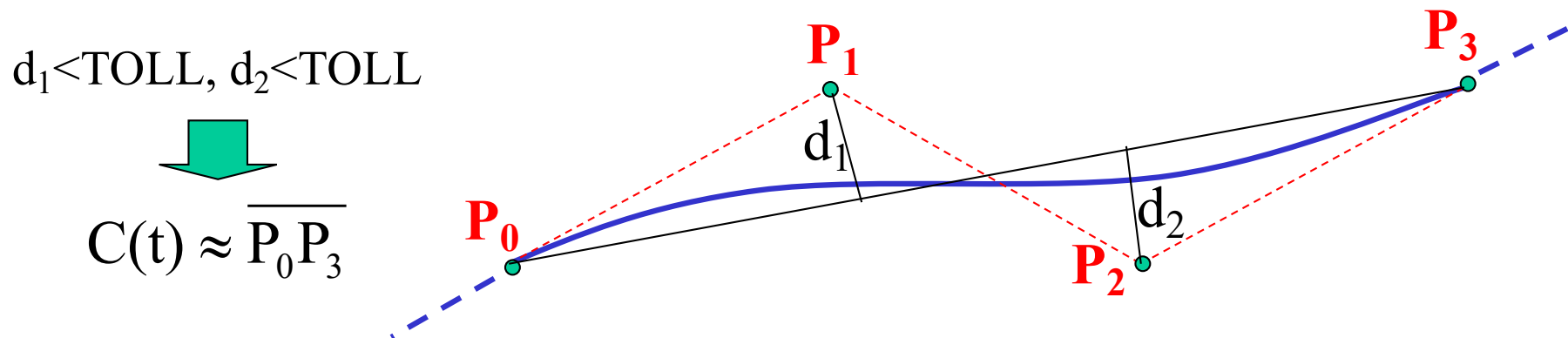
A) Metodo “uniforme”

- 1) Si valuta $C(t)$ per M valori equispaziati $t_1, \dots, t_M$ del parametro t
- 2) Si traccia la poligonale di vertici $C(t_1), \dots, C(t_M)$

B) Metodo “adattativo”

- 1) Si suddivide ricorsivamente $C(t)$
- 2) Si traccia la poligonale avente per vertici i punti di controllo dei segmenti di curva della suddivisione

La ricorsione è arrestata quando per un segmento di curva i punti di controllo ottenuti sono *sufficientemente vicini* al segmento stesso



Esercizio

1. Implementare l'algoritmo di de Casteljau per la valutazione di una curva di Bezier di grado n :

$$C(t) = \sum_{i=0}^n P_i B_{i,n}(t) \quad t \in [0,1]$$

2. Scrivere un programma (con OpenGL) per il rendering di una curva di Bezier di grado n mediante il metodo “uniforme”, utilizzando l'algoritmo implementato

1. Algoritmo di de Casteljau per la valutazione di una curva di Bezier di grado n

Assegnati: - i punti di controllo $P_0, \dots, P_n$
- un valore del parametro t in $[0,1]$

for $i=0, n$

$p_i^0 = P_i$

endfor

for $r=1, n$

for $i=0, n-r$

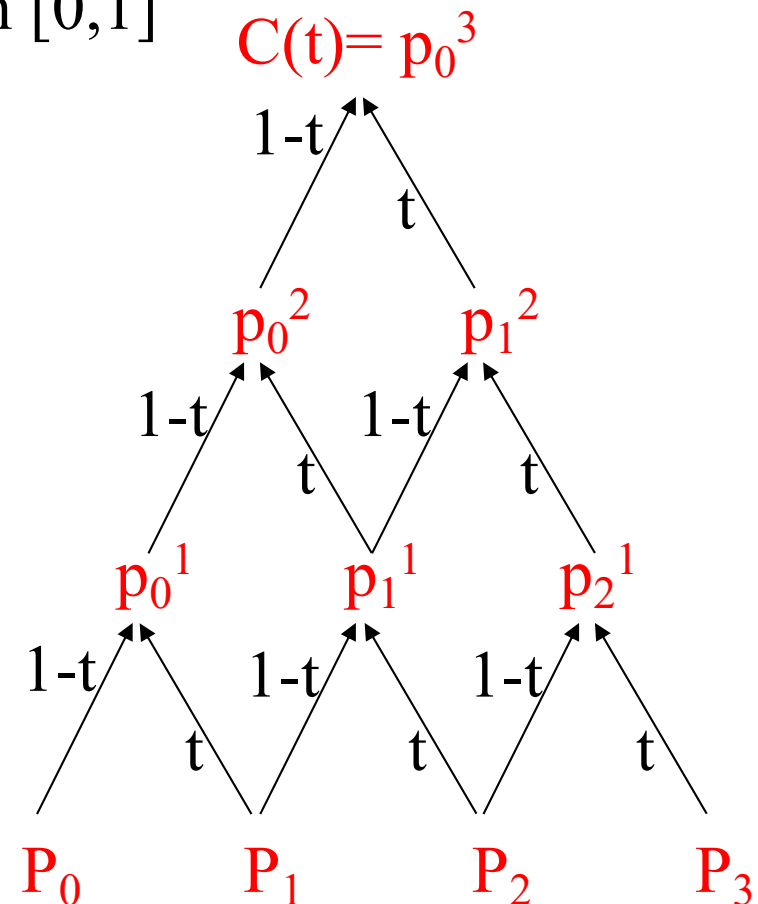
$p_i^r = (1-t) p_i^{r-1} + t p_{i+1}^{r-1}$

endfor

endfor

Risultato: $C(t) = p_0^n$

$T_{DC}(n) = O(n^2)$



2. Rendering di una curva di Bezier di grado n mediante il metodo “uniforme”

Metodo “uniforme”

- 1) Si valuta $C(t)$ per M valori equispaziati $t_1, \dots, t_M$ del parametro t
- 2) Si traccia la poligonale di vertici $C(t_1), \dots, C(t_M)$

Curve di Bezier in OpenGL

“Valutatori” (evaluators), definiti mediante:

glMap1f(type, u_min, u_max, stride, order, point_array);

- **type** = GL_MAP1_VERTEX_3 (tipo di oggetto da valutare)
- **u_min, u_max** = range del parametro
- **stride** = numero di valori f.p. fra un punto di controllo ed il successivo
- **order** = ordine della curva (= grado+1)
- **&point_array[0][0]** = puntatore al primo punto di controllo

Curve di Bezier in OpenGL (cont.)

Un valutatore deve essere prima abilitato:

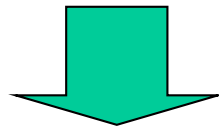
glEnable(type);

- **type** = tipo di oggetto da valutare.

Una volta definito ed abilitato un valutatore, per utilizzare un valore dal valutatore attivo si può usare:

glEvalCoord1f(u);

- **u** = valore del parametro in cui valutare l'oggetto



I vertici della curva possono essere usati come ogni altro vertice

Curve di Bezier in OpenGL: esempio

```
GLfloat cp[4][3] = { { -4.0, -4.0, 0.0},  
                    { -2.0,  4.0, 0.0},  
                    {  2.0, -4.0, 0.0},  
                    {  4.0,  4.0, 0.0}};
```

...

```
glMap1f(GL_MAP1_VERTEX_3, 0.0, 1.0, 3, 4, &cp[0][0]);  
glEnable(GL_MAP1_VERTEX_3);
```

...

```
glBegin(GL_LINE_STRIP);  
    for (i = 0; i <= 30; i++)  
        glEvalCoord1f((GLfloat) i/30.0);  
glEnd();
```

Curve di Bezier in OpenGL (cont.)

Se il valutatore attivo deve essere utilizzato per valutare l'oggetto per un insieme di m valori equispaziati del parametro, si può usare:

```
glMapGrid1f(m, u1, u2);  
glEvalMesh1(GL_LINE, i1, i2); /* o GL_POINT */
```

- $0 \leq i1, i2 \leq m$

in alternativa a:

```
glBegin(GL_LINE_STRIP); /* o GL_POINTS */  
    for (i = i1; i <= i2; i++)  
        glEvalCoord1f(u1+i*(u2-u1)/m);  
glEnd();
```


Curve di Bezier in OpenGL: esempio

```
GLfloat cp[4][3] = { { -4.0, -4.0, 0.0},  
                     { -2.0,  4.0, 0.0},  
                     {  2.0, -4.0, 0.0},  
                     {  4.0,  4.0, 0.0}};
```

...

```
glMap1f(GL_MAP1_VERTEX_3, 0.0, 1.0, 3, 4, &cp[0][0]);  
glEnable(GL_MAP1_VERTEX_3);
```

...

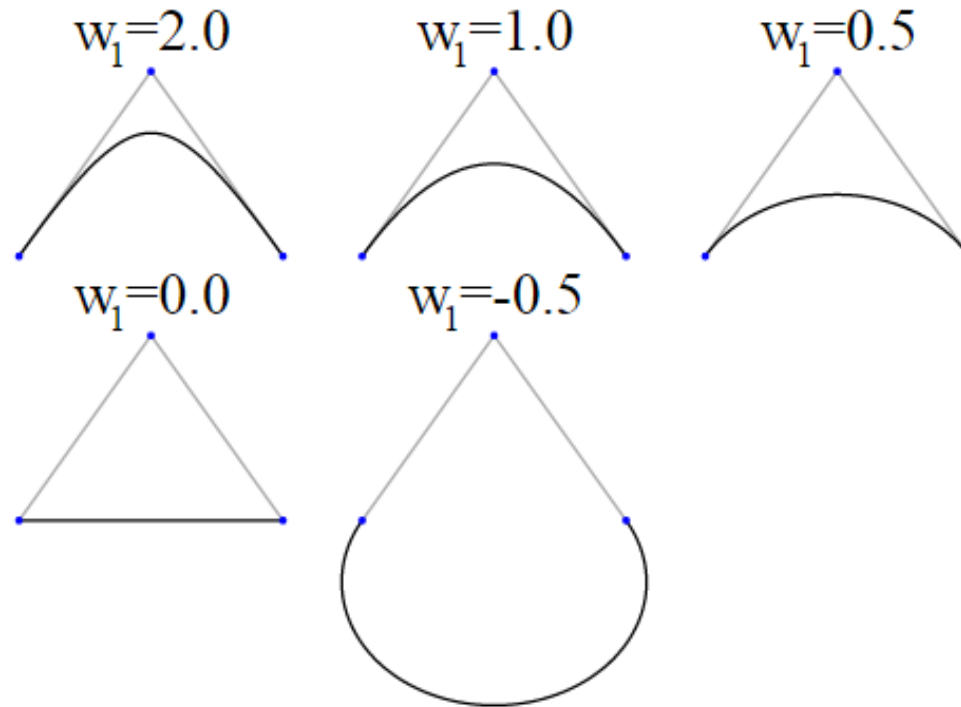
```
glMapGrid1f(30, 0, 1); /*inizializza i valori equispaziati*/  
glEvalMesh1(GL_LINE, 0, 30); /*disegna la curva*/
```

Esercizio

1. Scrivere un programma (con OpenGL) per il rendering di una curva di Bezier di grado n mediante il metodo “uniforme”, utilizzando
 - l’algoritmo di de Casteljau implementato e
 - le funzioni di OpenGL

Curve di Bezier razionali

$$C(t) = \frac{\sum_{i=0}^n w_i P_i B_{i,n}(t)}{\sum_{i=0}^n w_i B_{i,n}(t)} \quad t \in [0,1]$$



Curve di Bezier razionali

$$C(t) = \frac{\sum_{i=0}^n w_i P_i B_{i,n}(t)}{\sum_{i=0}^n w_i B_{i,n}(t)} \quad t \in [0,1]$$

Strategia generale per algoritmi relativi alle curve razionali

1. Calcolo delle coordinate omogenee P_i^W dei punti di controllo P_i
2. Applicazione dell'algoritmo alla curva non razionale con punti di controllo P_i^W
3. Proiezione (omogeneizzazione) del risultato

Curve di Bezier razionali in OpenGL: esempio

```
GLfloat cp[4][3] = {      { -4.0, -4.0, 0.0},  
                        { -2.0,  4.0, 0.0},  
                        {  2.0, -4.0, 0.0},  
                        {  4.0,  4.0, 0.0}};
```

```
GLfloat w[4] = { 1.0, 5.0, 1.0, 1.0};
```

```
GLfloat cpw[4][4];
```

...

“Calcolo cpw (coordinate omogenee dei cp)”

...

```
glMap1f(GL_MAP1_VERTEX_4, 0.0, 1.0, 4, 4, &cpw[0][0]);
```

```
glEnable(GL_MAP1_VERTEX_4);
```

...

```
glMapGrid1f(30, 0, 1); /*inizializza i valori equispaziati*/
```

```
glEvalMesh1(GL_LINE, 0, 30); /*disegna la curva*/
```

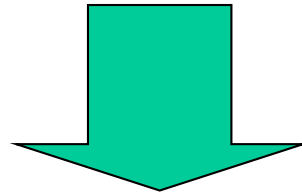
Osservazione

- Finora $C(t)$ è stata definita per $t \in [0,1]$ (per semplicità)
- Per rappresentare la stessa curva $C(t)$ con intervallo di parametrizzazione $[a,b]$:

$$C(t) = C\left(\frac{s-a}{b-a}\right), \quad s \in [a, b]$$

Svantaggi delle curve di Bezier

- grado = numero dei punti di controllo -1
- controllo globale



Curve composite
(unione di più curve)

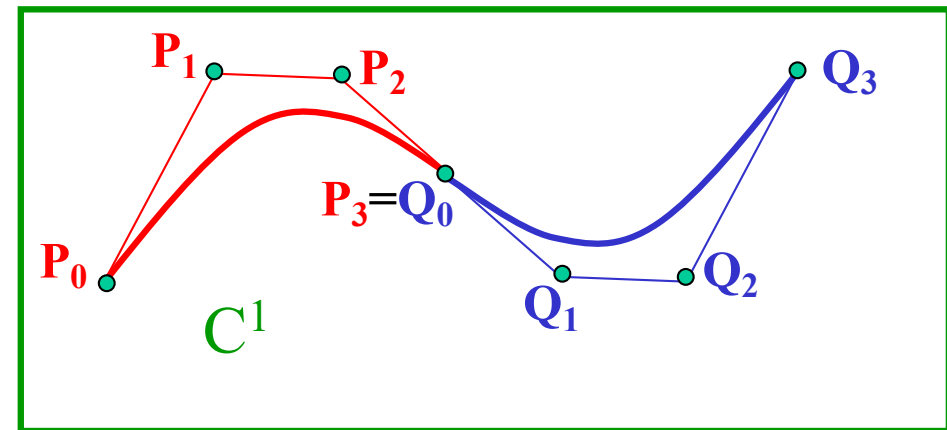
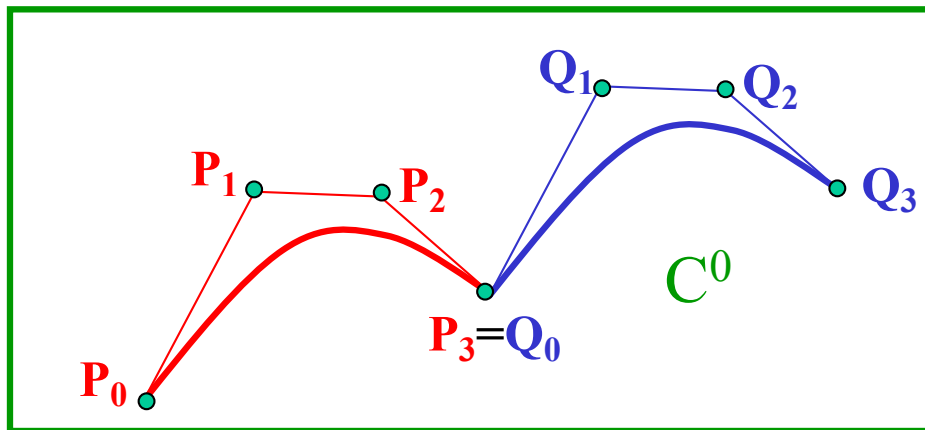
Composizione di due curve di Bezier

$C_A(t)$ con punti di controllo P_0, P_1, P_2, P_3

$C_B(t)$ con punti di controllo Q_0, Q_1, Q_2, Q_3

$P_3 = Q_0 \longrightarrow C_A(t) \cup C_B(t)$ ha continuità C^0

$P_3 - P_2 = Q_1 - Q_0 \longrightarrow C_A(t) \cup C_B(t)$ ha continuità C^1



Per costruire una curva composta con continuità C^1 basta scegliere allineati ed equidistanti i 3 punti $P_2, P_3 = Q_0, Q_1$

Esercizio Avanzato

1. Scrivere un programma (con OpenGL) per il rendering di curve di Bezier composite, consentendo l'input dei punti di controllo mediante mouse